A BUILD-ALONG WORKSHOP · 3 HOURS

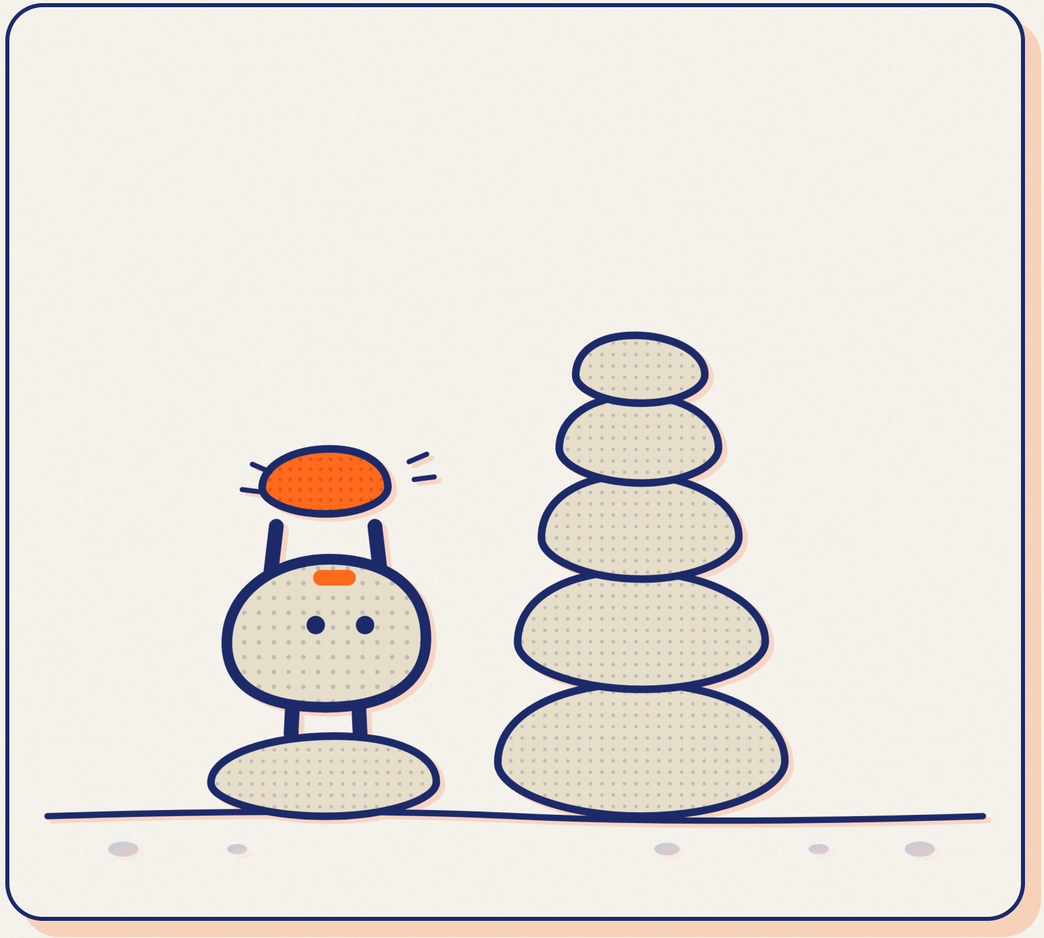
Stone by stone

Build a personal site you will still be improving a year from now — from a one-page idea to your own domain, in one session.

Claude · Claude Code · GitHub · a web framework · Cloudflare

SANKET TAMBARE · SANKETTAMBARE.IN

PRESS → TO BEGIN · N FOR NOTES · F FOR FULL SCREEN



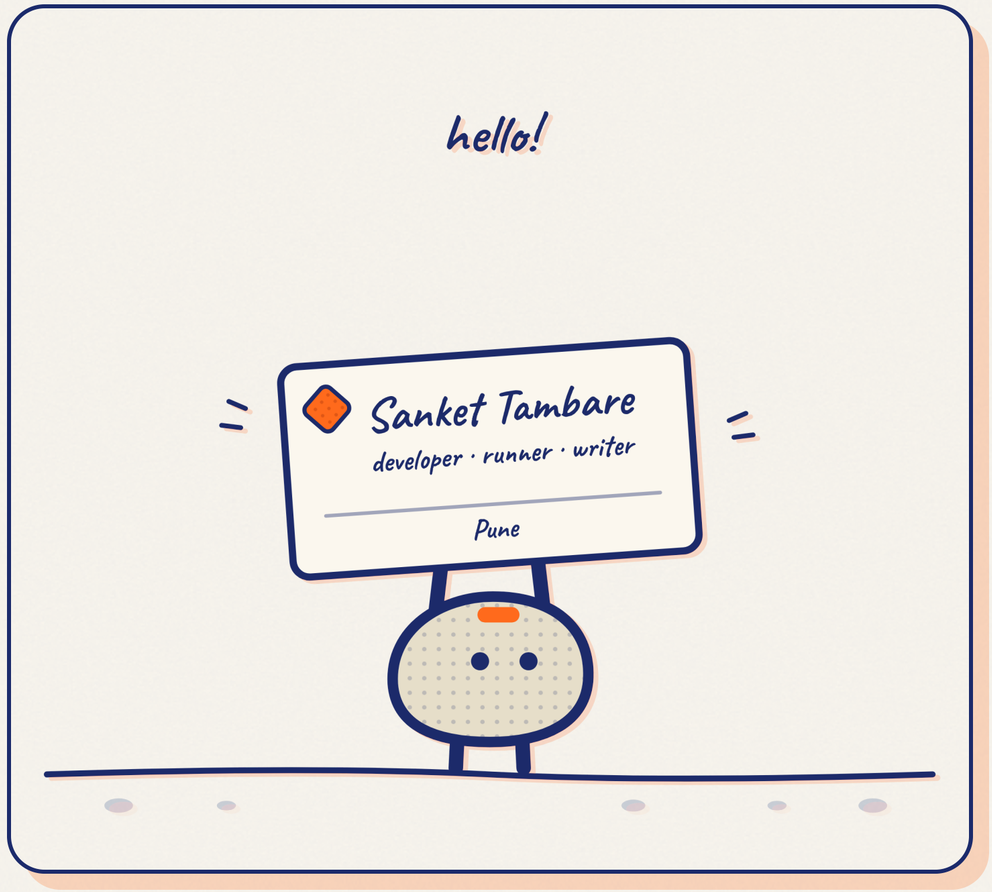
00 · WELCOME 0:01

Hi, I'm Sanket

Software engineer at **Bridgenext** in Pune — full-stack development, data pipelines and DORA metrics. Computer science at Rajarambapu Institute of Technology; technology policy at the Takshashila Institution, and a NAST fellow working on AI governance.

- Endurance runner — 10K races to a 50K ultra
- Trekker of the Sahyadri forts
- Writes in English and Marathi — essays, a monthly newsletter, the *Dare Write's* podcast
- Builds sites for causes: NGOs, a birding marathon, a data story on the Sahyadri

`sankettambare.in · card.sankettambare.in`



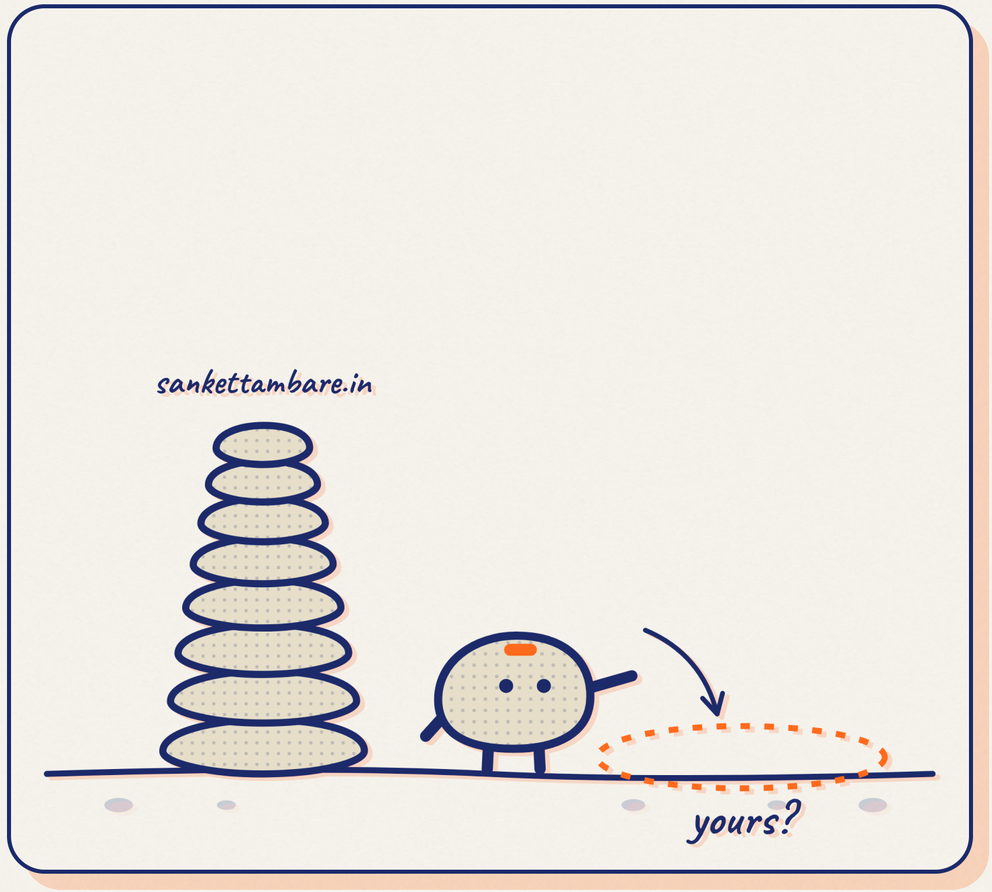
00 · WELCOME 0:03

Why this workshop

When I showed you my site, the response was the same every time: *“I want one of these.”*

- **Building got cheap.** An AI coding agent writes the code; you bring the judgment.
- **Keeping it alive didn't.** Most sites built in an afternoon are never touched again.
- **You already have the hard skill.** Scoping, phasing, cutting, checking — that's project management, and it's exactly what this needs.

> **Today isn't about a website. It's about a way of building you can repeat.**

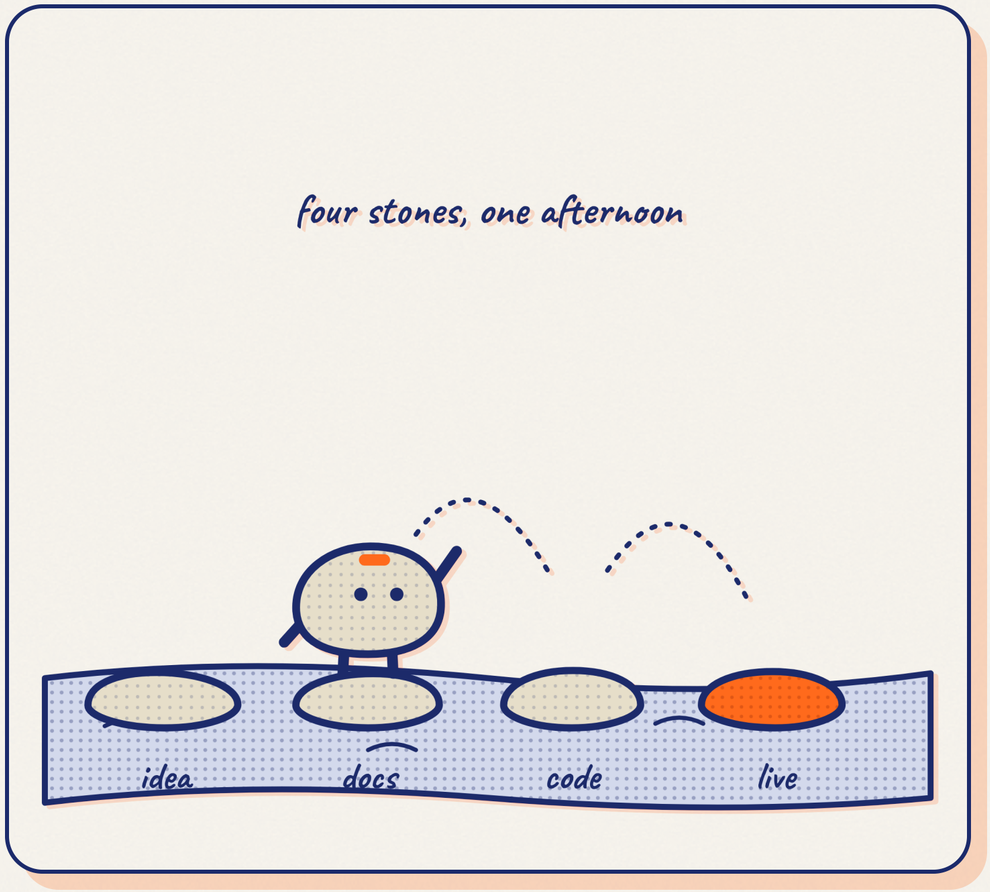


00 · WELCOME 0:05

What we'll learn today

1. **Shape an idea** with Claude — an interview, deep research, three short documents
2. **Choose a stack you own** — and know the pros and cons of each choice
3. **Keep data apart from code**, so either can change without the other
4. **Build in phases** with Claude Code — one session each, checked before it counts
5. **Ship it** — GitHub, Cloudflare and your own domain
6. **Keep it growing** — every later change is a prompt

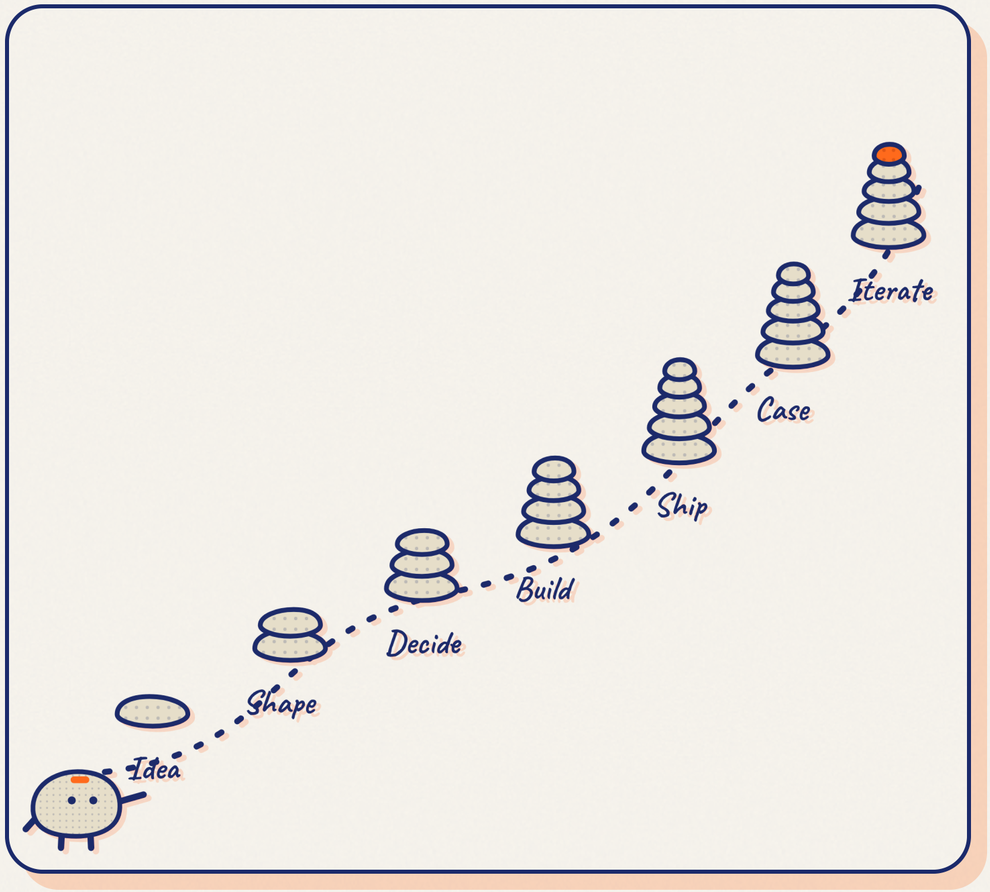
✓ YOU SHOULD NOW HAVE by 3:00 — a live URL, a GitHub repo, three planning docs and the prompt pack to keep going.



01 · THE IDEA 0:07

The three hours

0:00	**Welcome** — who I am, why this, what you'll learn
0:08	**Idea** — why own the stack, what it is
0:20	**Shape** — idea.md → interview → deep research → three docs → data
1:05	**Decide** — framework and host, pros and cons
1:20	*Break — 10 minutes*
1:30	**Build** — repo, data apart from code, one phase per session, audits
2:20	**Ship** — GitHub → Cloudflare → your domain
2:38	**Case study** — sankettambare.in and four featured projects
2:52	**Iterate** — then your feedback, and thank you



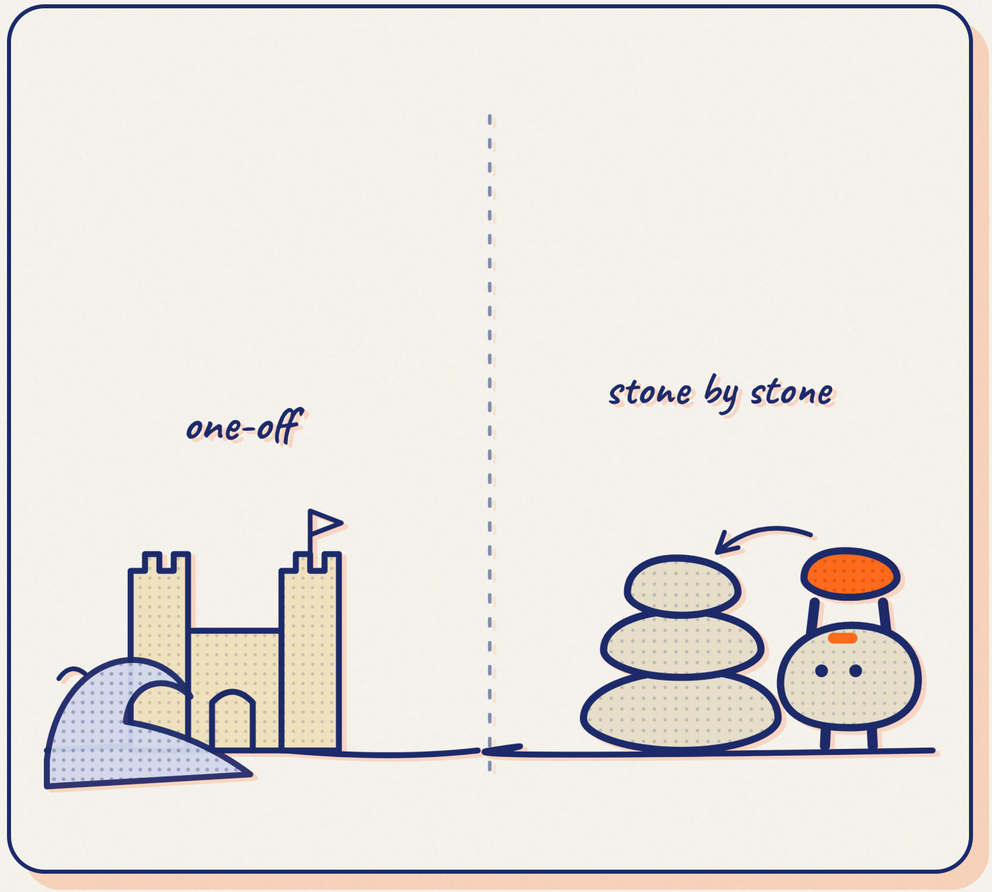
01 · THE IDEA 0:08

Sandcastles and cairns

Lovable, Bolt and v0 build a beautiful sandcastle in an afternoon. Then the tide comes: the idea changes, credits run out, the code lives in someone else's editor — and nobody adds a second wing.

A cairn is built stone by stone. Each visit adds one, and it stands because every stone sits on the last.

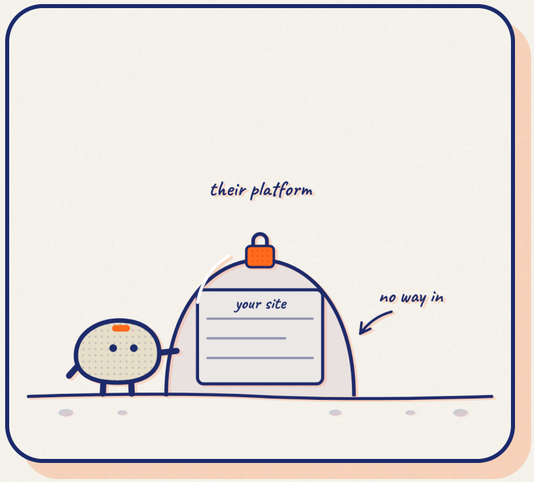
> **The question isn't “can I build it today?” It's “will I still improve it tomorrow?”**



01 · THE IDEA 0:10

The iteration test

ASK OF ANY SITE	APP BUILDER	YOUR OWN STACK
Where does the code live?	In the platform (export is possible, then you're on your own)	Your GitHub repo — every version, forever
Where does the content live?	Mixed into the generated components	Plain files in `content/`, apart from the code
Change one word — how long?	Open the builder, prompt, hope nothing else moves	Edit a file or ask Claude Code; live in a minute
What does keeping it cost?	A monthly plan or credits	Free tiers cover a personal site
Who can work on it?	That platform's agent	Any agent, any developer, you
What's the history?	A chat log	Git log + a changelog of every change



IN PLAIN WORDS **Export** download the code to run somewhere else · **changelog** a dated list of every change.

01 · THE IDEA 0:12

The stack: four stones

THE BUILDER

Claude Code

Reads the repo, writes the code, runs the build, checks its own work. In a terminal, an IDE, the desktop app, the browser or your phone.

THE MEMORY

GitHub

Every change is a commit: history, review, rollback. A push is also what triggers a deploy.

THE SHAPE

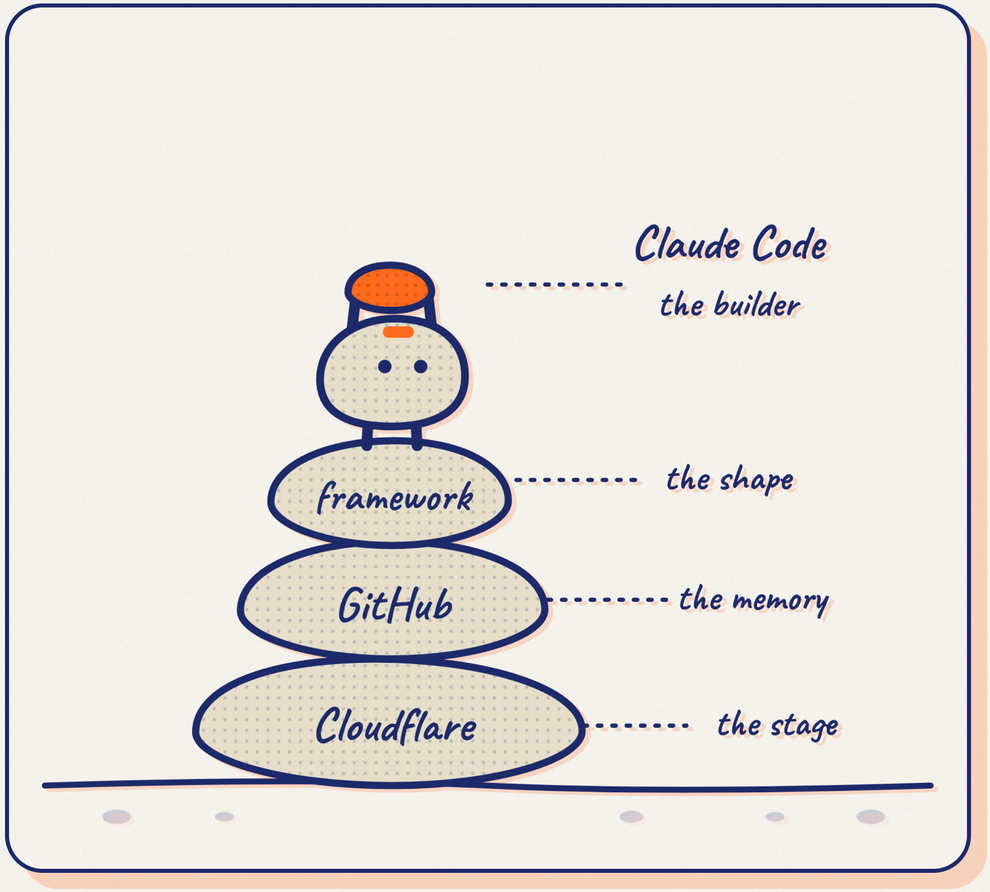
A framework

Pages, components, content collections — conventions an agent already knows. Astro is our default; slide 14 compares it.

THE STAGE

Cloudflare

Builds on every push, serves from the edge, and holds the domain, DNS and SSL in one place. Generous free tier.



IN PLAIN WORDS **Claude Code** an AI that edits your project's files · **GitHub** online storage that keeps every version · **framework** a starter kit for pages · **Cloudflare** the servers that show your site to the world.

01 · THE IDEA 0:14

“Scalable” means it compounds

Not traffic. Scalable here means the hundredth change costs no more than the first — small, reviewed, reversible, live in a minute.

31

major versions of sankettambare.in — each a new page or a redesign

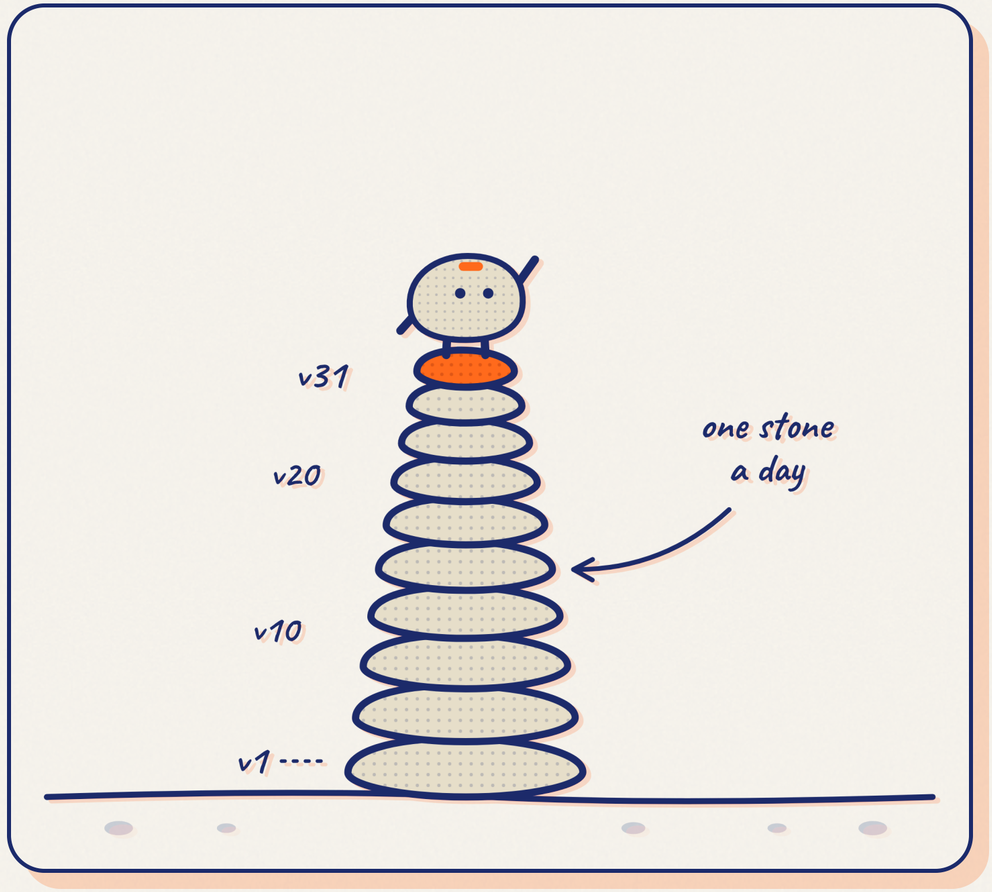
43

routes, each with its own share card

1

stone a day — a typo, a photo, a page

Numbers from the case-study repo's route manifest and changelog.



IN PLAIN WORDS **Major version** a release big enough to be a new edition — a new page or a redesign · **route** one page address, like /about.

01 · THE IDEA 0:16

Before you start

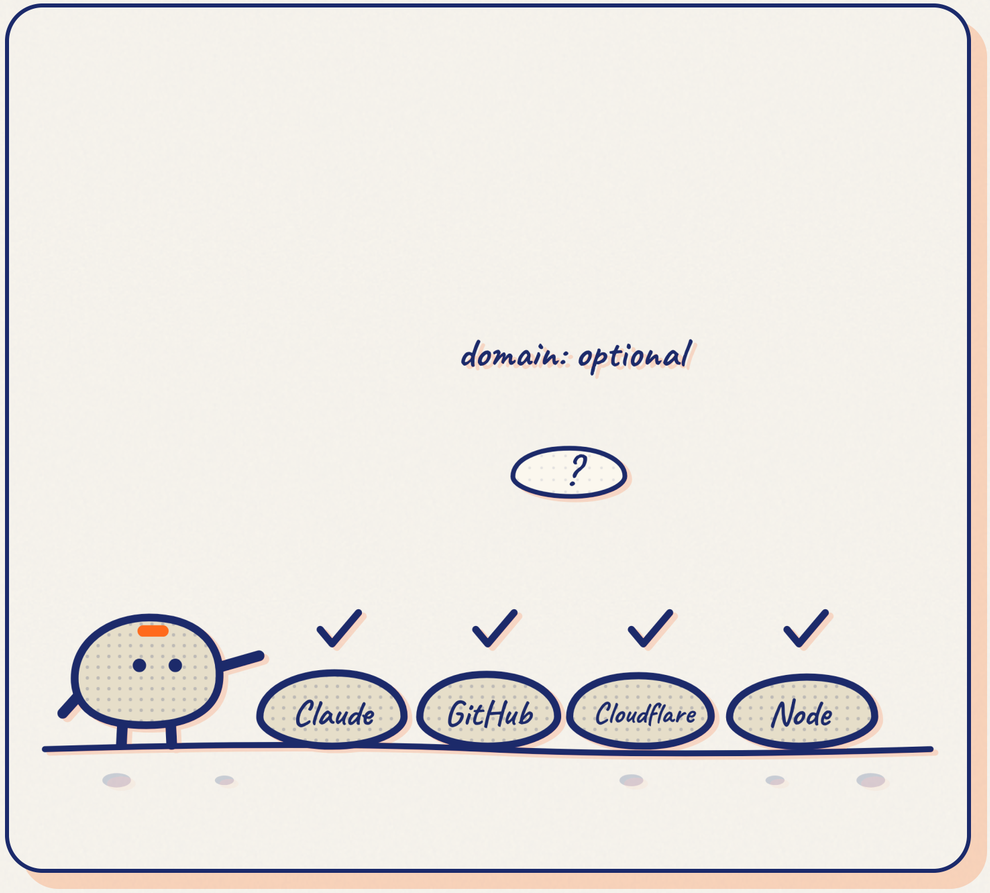
Accounts

- **Claude** on a paid plan — chat with Research, plus Claude Code
- **GitHub** — free
- **Cloudflare** — free plan
- A **domain** — optional today; buy it at Cloudflare or any registrar

On your laptop

- **Node.js** LTS and **Git**
- **Claude Code** — install from code.claude.com
- An editor (VS Code is fine)
- A folder of your stuff: bio, photo, CV, project links

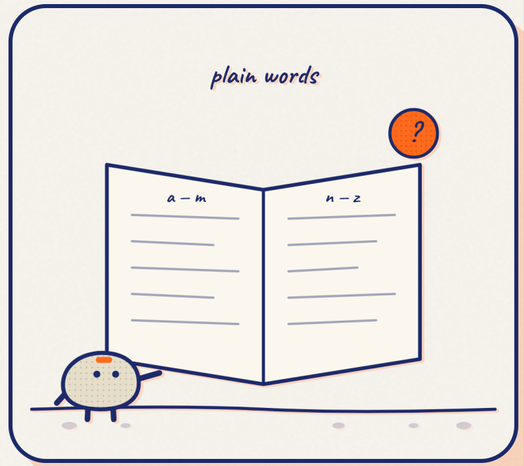
✓ YOU SHOULD NOW HAVE four accounts signed in, `claude` runs in a terminal. No laptop setup? Claude Code on the web runs in the cloud at claude.ai/code.



IN PLAIN WORDS **Node.js** the engine that runs website tools on your laptop · **Git** the tool that records versions · **terminal** a window where you type commands.

Words we'll use today

No jargon needed. On every later slide, a dotted word shows this meaning when you hover over it or tap it.



Repository (repo)
A project folder that remembers every version of every file — like a Google Doc's version history, for a whole folder.

Commit
A saved checkpoint with a short note saying what changed.

Push
Upload your saved checkpoints from your laptop to GitHub.

Branch
A safe copy of the project to try a change without touching the live site.

Pull request
A page asking to merge a branch back in, where the change can be reviewed first.

Framework
A starter kit with rules for building pages, so you don't start from a blank file. Astro is one.

Build
Turning your source files into the finished website files.

Deploy
Putting the built website on the internet for everyone.

Host / hosting
The computers that send your site to each visitor — like renting shop space.

Edge
Copies of your site kept in cities around the world, so it loads fast wherever the visitor is.

Domain
Your site's name, like yourname.com.

DNS
The internet's phonebook: it turns a name like yourname.com into the address of a computer.

Nameservers
Whoever keeps your domain's phonebook entry. Pointing them at Cloudflare lets Cloudflare manage it.

SSL / HTTPS
The padlock in the address bar: traffic between visitor and site is encrypted.

Context
How much of the conversation Claude can keep in mind at once. Long chats crowd it out.

Plan mode
Claude writes a plan and waits for your OK before it changes any file.

CLAUDE.md
A short note in your project that Claude reads at the start of every session — its onboarding doc.

Data layer / code layer
What the site says (words, photos, links) versus how it looks and works. Keep them in separate folders.

Markdown
Plain text where # makes a heading and - makes a bullet. Easy for people and AI to read.

JSON / YAML
Plain-text formats for lists of things — like a spreadsheet written as text.

SEO
Search engine optimisation: helping Google and AI search find and show your pages.

Lighthouse
Google's free report card for a page: speed, accessibility, SEO and best practice, each out of 100.

Open Graph image
The preview picture shown when your link is shared on WhatsApp, LinkedIn or Slack.

Terminal
A window where you type commands to your computer instead of clicking icons.

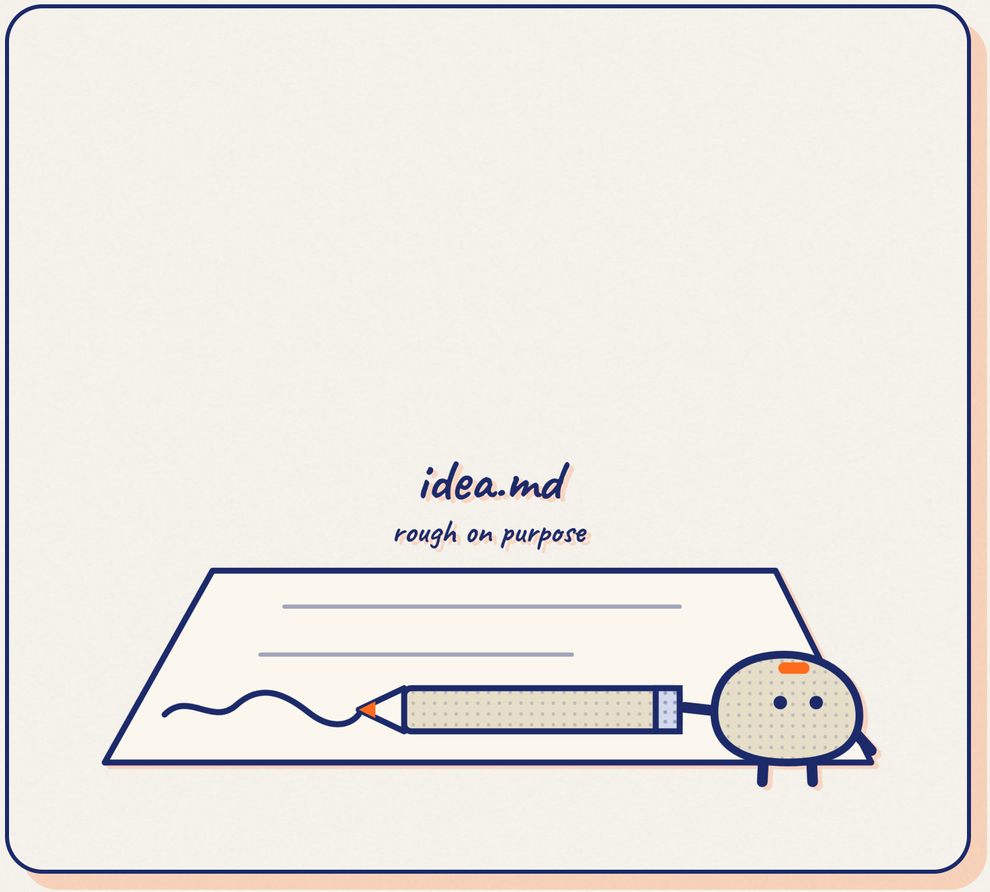
02 · SHAPE 0:20 · EXERCISE, 10 MIN

Write idea.md

One page, in your own words, before any AI. It's rough on purpose — the next step sharpens it.

```
# idea.md
Who is it for?            one sentence
What should they do?      the one action — hire, read, contact
Three things it must do
What I already have       posts, photos, CV, projects
What it must never be     corporate? cluttered? slow?
Time I'll give it         per week, honestly
```

✓ YOU SHOULD NOW HAVE **idea.md saved — a Google Doc or a text file is fine.**



02 · SHAPE 0:30 · CLAUDE CHAT

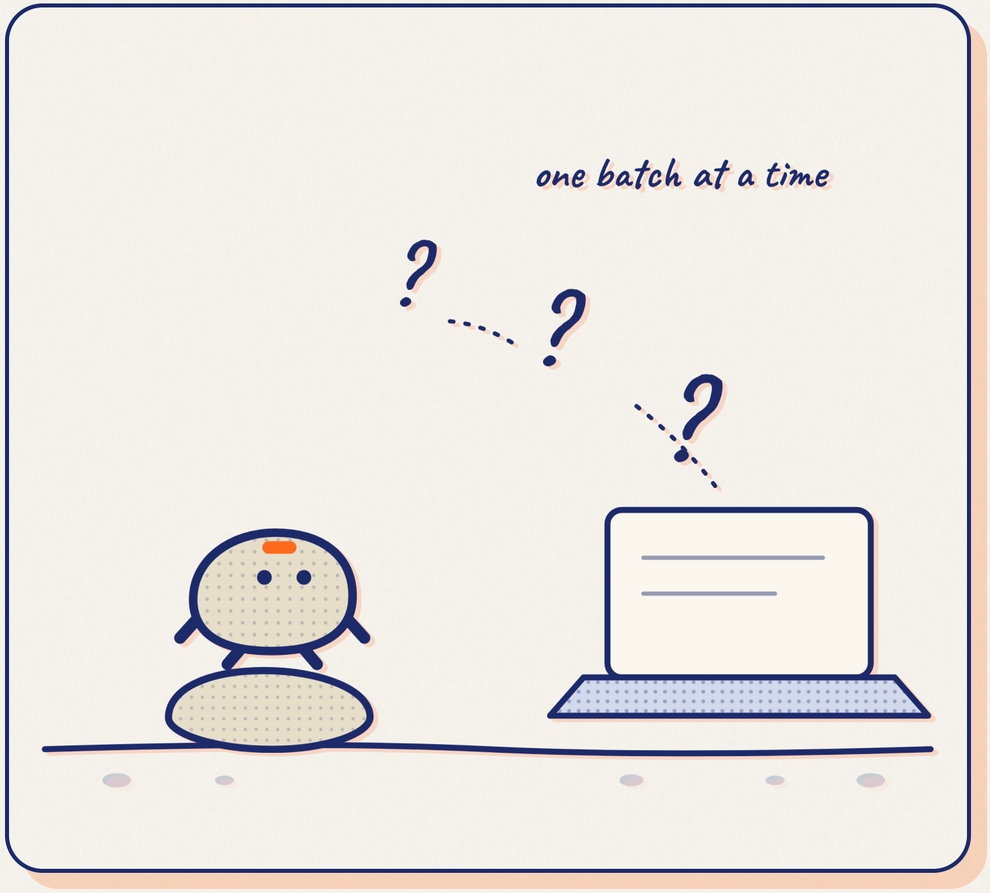
Let Claude interview you

PROMPT · CLAUDE CHAT

```
Here is idea.md, a rough idea for my personal website. Before you
write anything, interview me. Ask at most 5 questions at a time,
most important first: audience, the one thing a visitor should do,
content I already have, tone, what it must never be, and time I can
give it each week.

Keep asking until you could write the concept without guessing.
Then rewrite it as concept.md: problem, audience, goals, non-goals,
pages, content sources, how I'll know it works. Under 600 words.
```

Paste idea.md under it. Start a Claude Project for this site so the documents stay together.



IN PLAIN WORDS **Claude Project** a folder in Claude chat that keeps the files and conversations for one thing together.

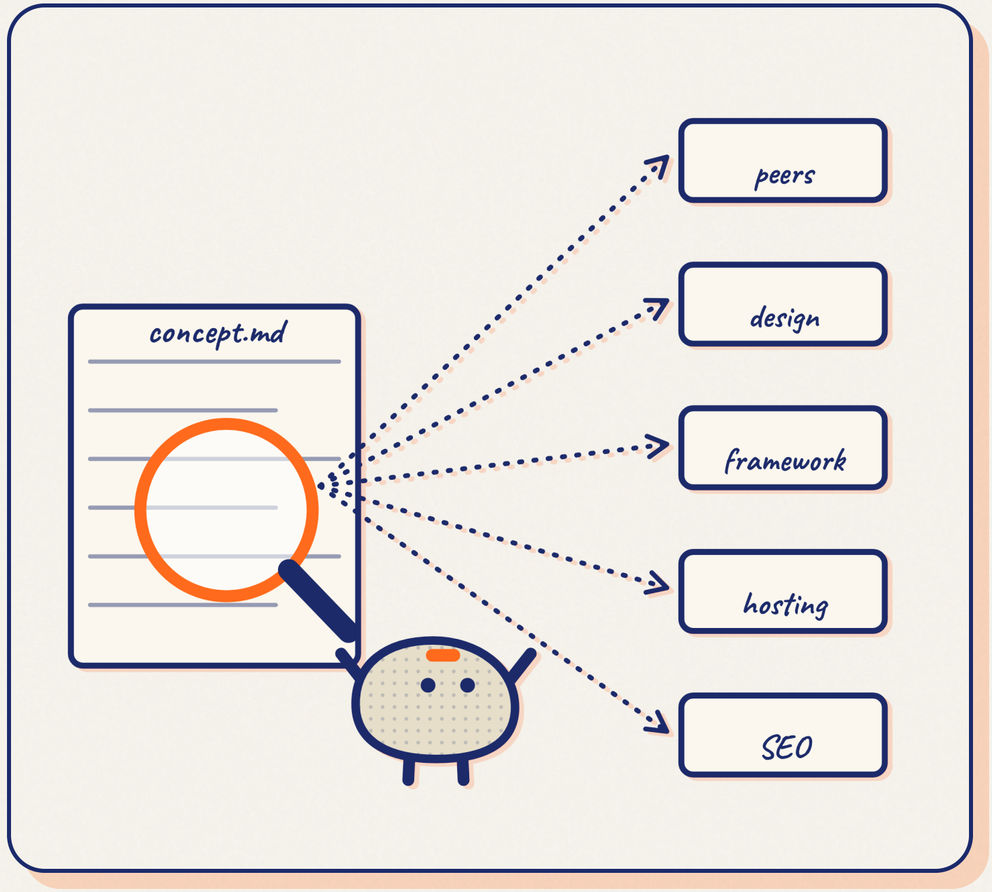
02 · SHAPE 0:40 · RESEARCH ON

Deep research, one aspect at a time

PROMPT · CLAUDE CHAT, RESEARCH ON

```
Using concept.md, research each aspect separately:
1. 3–5 well-regarded personal sites in my field — what makes each
work
2. Design patterns: homepage structure, navigation, project case
studies, type and colour systems
3. Frameworks for a content-first site in 2026 — Astro, Next.js,
SvelteKit, React + Vite — for someone iterating with an AI coding
agent
4. Hosting: Cloudflare vs alternatives — limits, pricing, custom
domains
5. SEO, accessibility and performance baselines
Cite every claim with a link. Flag what you could not verify.
```

Turn Research on from the chat box's tools menu. If one aspect comes back thin, re-run it on its own.



IN PLAIN WORDS **Research** a Claude mode that reads many pages for minutes and writes a cited report · **SEO** helping Google find your pages.

Three documents out

WHY + WHAT

concept.md

Audience, goals, non-goals, pages, success.

HOW

technical-decisions.md

Each decision: options, pros/cons, choice, when to revisit.

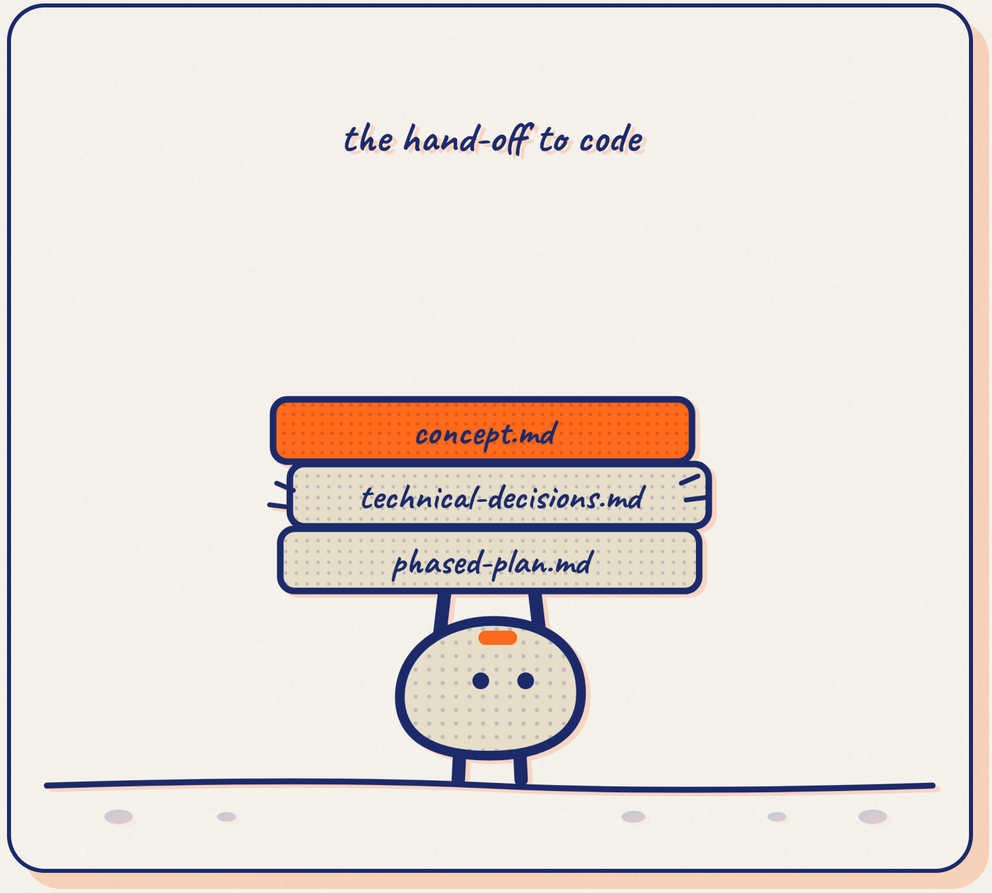
WHEN

phased-plan.md

Phase 0 setup, Phase 1 live, then ≤ 5 features a phase.

PROMPT · CLAUDE CHAT

```
From concept.md and the research, write three files. 1) concept.md, final. 2) technical-decisions.md: one entry per decision — framework, hosting, where content lives (kept apart from code), styling, images, SEO, analytics — with options, pros, cons, choice, and what would make us revisit it. 3) phased-plan.md: Phase 0 setup, Phase 1 the smallest site that can go live, later phases of at most 5 one-line features, each with how to verify it and what is out of scope.
```



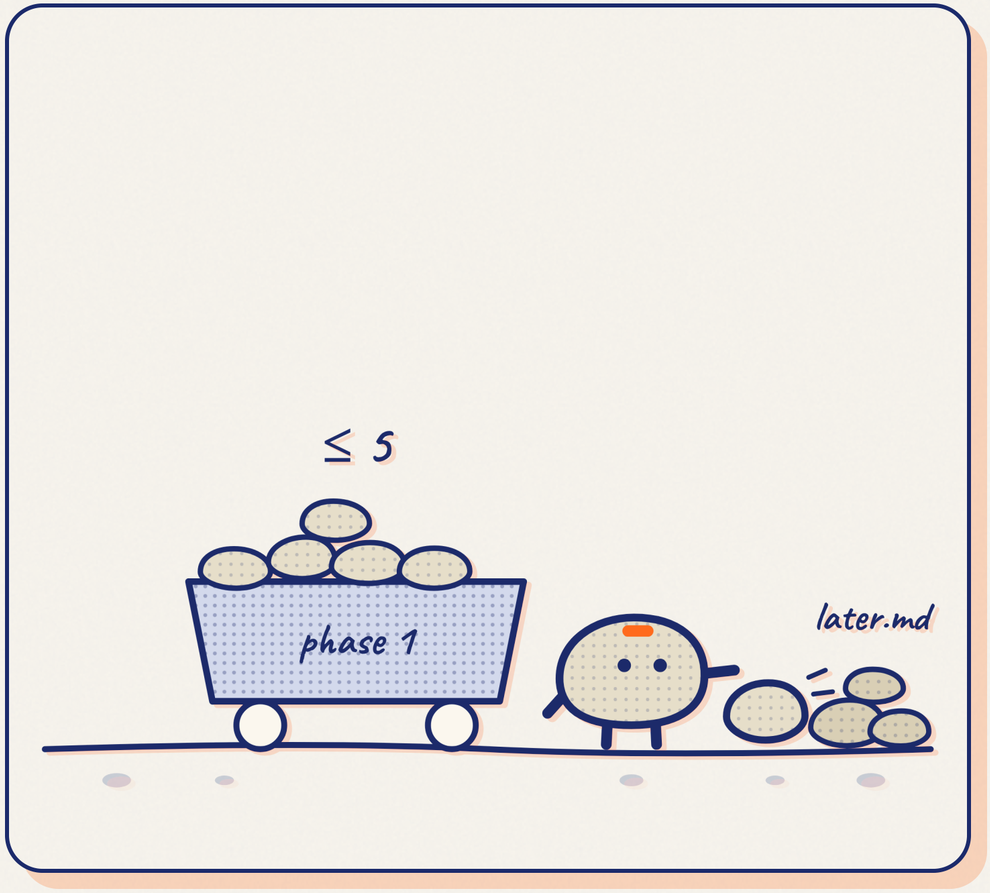
IN PLAIN WORDS **.md (Markdown)** a plain-text file where # makes a heading and - makes a bullet — readable by people and AI alike.

02 · SHAPE 0:57

Be brief: the plan is a list of cuts

- **One line per feature.** If it needs a paragraph, it's two features.
- **Phase 1 is the smallest site that can go live** — home, about, work, contact.
- **At most five features a phase**, and every phase ends deployed.
- New ideas go to `later.md`, never into the current phase.

> **A long feature list is a wish. A short one is a plan.**



IN PLAIN WORDS **Phase 1** the smallest version that is useful and live · **later.md** a parking lot for ideas.

02 · SHAPE 1:00

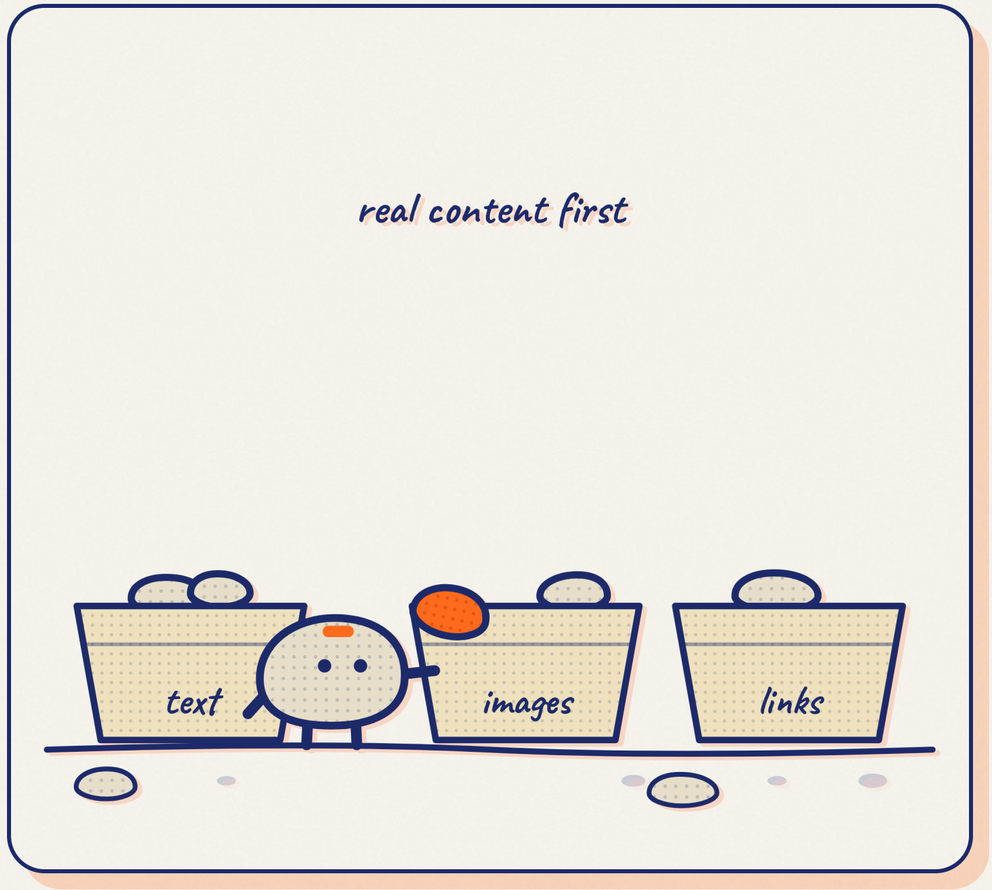
Gather all the data before any code

An agent designs around the content it's given. Real titles, real photos and the real number of projects make a layout that fits; lorem ipsum makes one that breaks the day you fill it.

- Bio in three lengths
- Photos — originals
- Projects: title, one line, link, image
- Writing, talks, CV, social links

FORMATS

Prose → Markdown with frontmatter
Lists → JSON or YAML
Images → `content/images/`



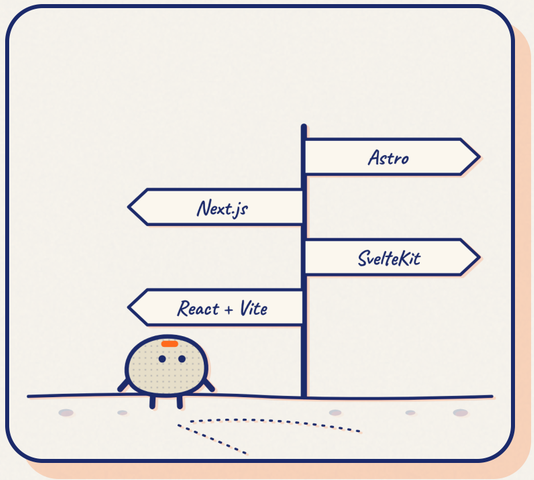
✓ YOU SHOULD NOW HAVE a content folder, and `INVENTORY.md` listing what each page needs — ask Claude to write it from concept.md.

IN PLAIN WORDS **Markdown** plain text for writing · **JSON / YAML** plain text for lists · **frontmatter** labelled lines at the top of a file (title, date).

03 · DECIDE 1:05

Choosing the framework

FRAMEWORK	FOR	AGAINST
Astro CONTENT FIRST	+ Ships no JavaScript unless a part needs it + Content collections: typed, validated Markdown + Cloudflare bought the Astro team (Jan 2026); first-class adapter	– Not for app-heavy dashboards – Smaller ecosystem than React
Next.js APP FIRST	+ Biggest ecosystem; full-stack React + Most examples for an agent to learn from	– Heavier pages, more concepts (server vs client) – Smoothest on Vercel; Cloudflare via an adapter
SvelteKit POLISHED, LIGHT	+ Small bundles, simple syntax + Official Cloudflare adapter	– Smaller ecosystem – Fewer examples to copy
React + Vite THE CASE STUDY	+ Simple mental model, huge ecosystem + Fine for very interactive sites	– Renders in the browser: crawlers see a blank page unless you add a fallback – More JavaScript per page

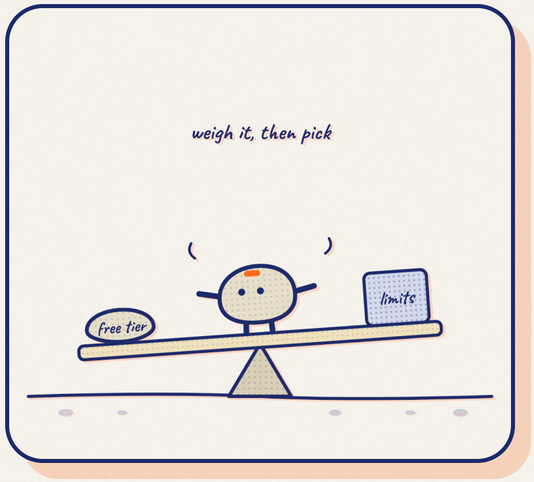


IN PLAIN WORDS **JavaScript** code that runs in the visitor's browser — less is faster · **crawler** a robot that reads sites for search engines and AI.

03 · DECIDE 1:12

Choosing the host

HOST	FOR	AGAINST
Cloudflare OUR PICK	+ Static requests free and unlimited on Workers + Domain, DNS, SSL and edge functions in one place + Commercial use allowed on the free plan	– Free Workers allow 10 ms CPU per request – Pages vs Workers is confusing — new projects: Workers
Vercel	+ Best Next.js experience; preview per PR	– Free Hobby plan is non-commercial – Pro is per seat, per month
Netlify	+ Simple; built-in forms	– Credit-based free plan; sites pause at the limit
GitHub Pages	+ Free, zero setup, beside the code	– Static only — no functions, few headers or redirects



Free-tier terms change — check each vendor's pricing page the day you choose.

IN PLAIN WORDS **Host** the computers that serve your site · **edge** copies of it in cities near visitors · **CPU limit** how long your code may think per visit.

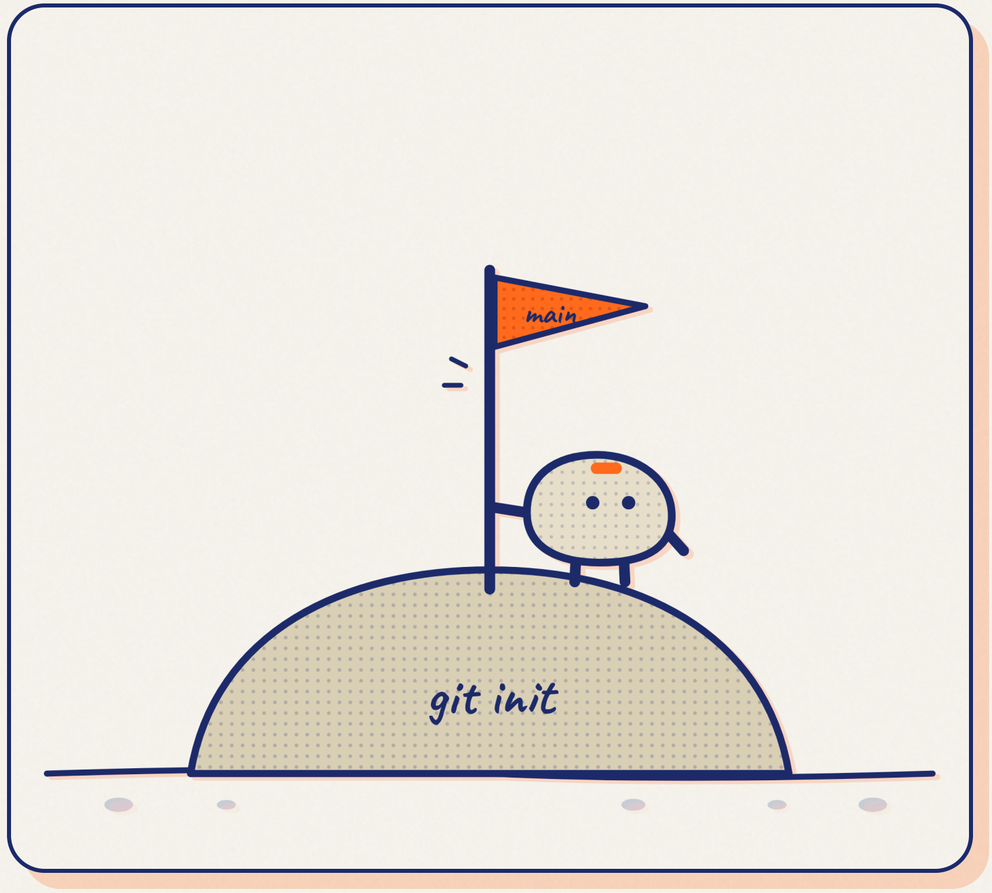
04 · BUILD 1:30 · CLAUDE CODE

Phase 0: the repo

PROMPT · CLAUDE CODE

```
Read docs/technical-decisions.md. Do
Phase 0: git init on main, scaffold
the framework we chose, create
content/ from content/INVENTORY.md
with my real content (no lorem
ipsum), a CLAUDE.md under 30 lines,
and CHANGELOG.md. Make sure the build
passes, then commit "Phase 0".
```

```
my-site/
├─ content/    what it says
│  ├─ profile.json
│  ├─ projects/*.md
│  └─ images/
├─ src/        how it looks
│  ├─ lib/content
│  ├─ components/
│  └─ pages/
├─ docs/       the 3 docs
├─ CLAUDE.md
└─ CHANGELOG.md
```



✓ YOU SHOULD NOW HAVE a folder with the three docs in docs/, Phase 0 committed. Then on github.com: New repository (empty) and run the two push lines it shows.

IN PLAIN WORDS **Repo** a project folder that remembers every version · **commit** a saved checkpoint with a note · **scaffold** the starter files a framework creates.

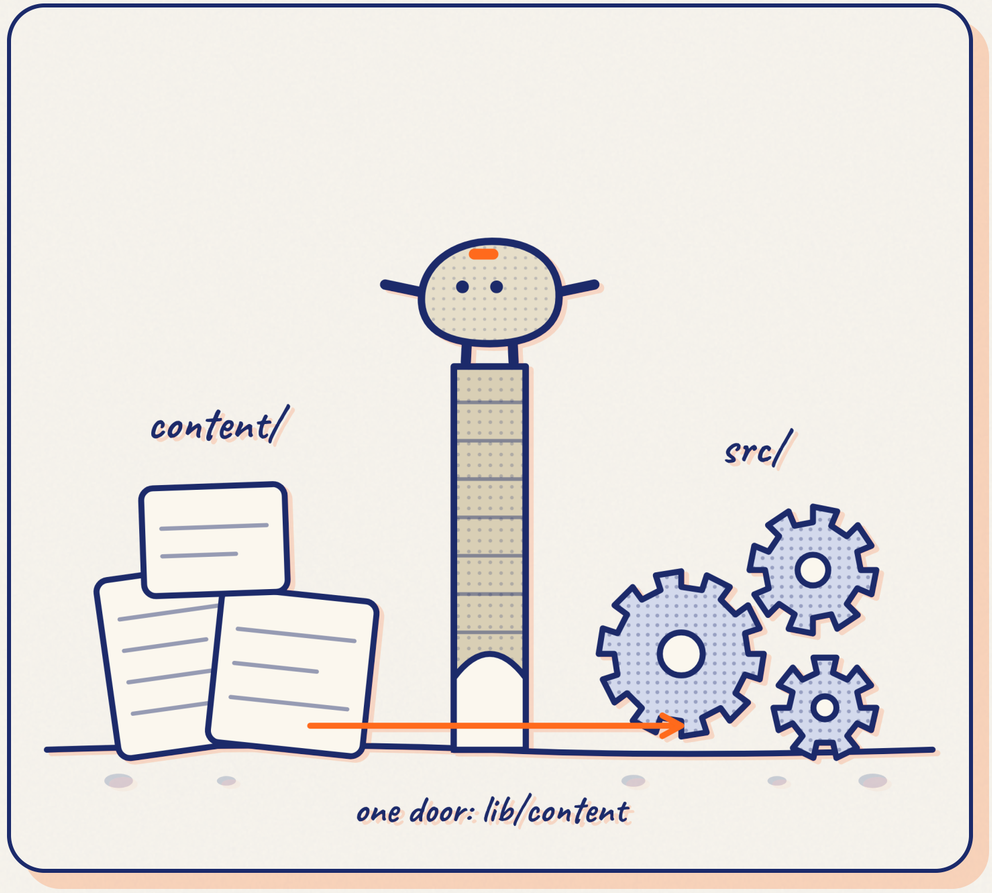
04 · BUILD 1:40

Keep the data layer apart from the code layer

1. **Copy, links and numbers live in** `content/` — never typed into a component.
2. **One module reads content** (`src/lib/content`). Components get data as props.
3. **Content has a schema** — Astro's content collections reject a project with no title.

Then you can change words without touching code, redesign without touching words — and later swap files for a CMS or a database by rewriting *one* module.

The case study did exactly that: its content moved from Markdown files to a database, and the page components never noticed.



IN PLAIN WORDS **Data layer** what the site says · **code layer** how it looks and works · **schema** rules content must follow, e.g. every project has a title.

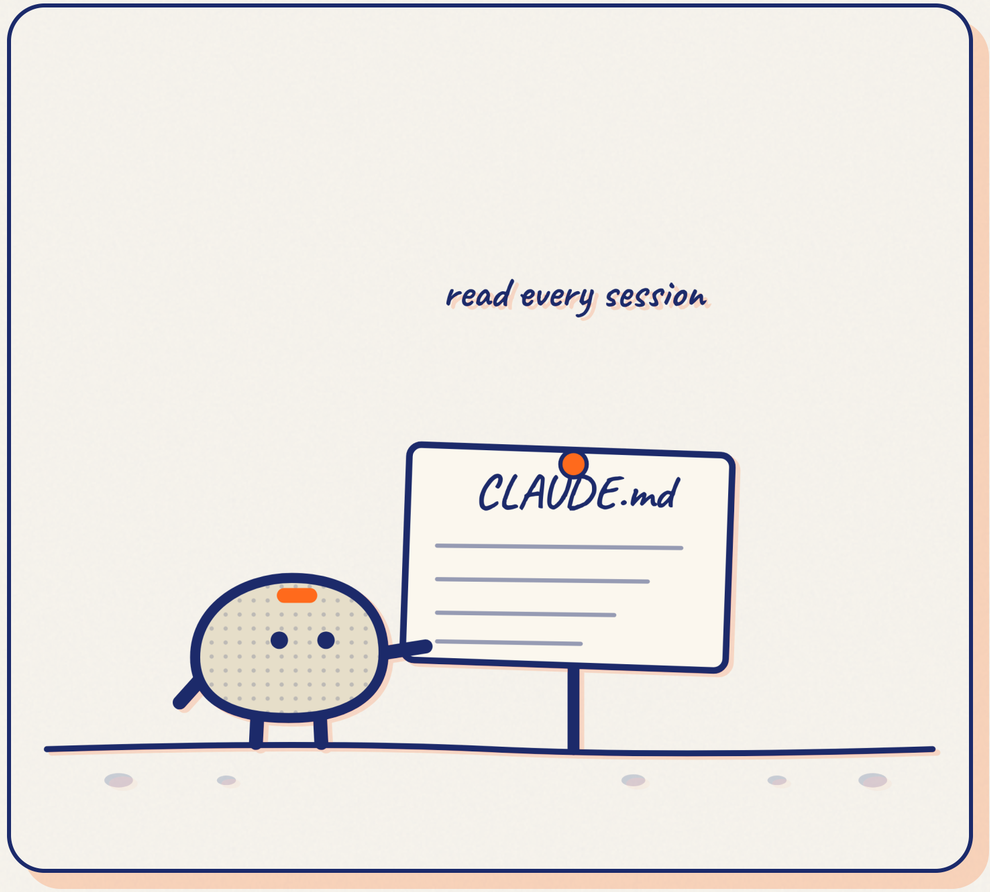
04 · BUILD 1:45

CLAUDE.md: the project's memory

Claude Code reads it at the start of every session. Keep it short: only what Claude can't work out from the code.

```
# my-site
Personal site. Astro → Cloudflare Workers.
- Content lives in content/; only src/lib/content
  reads it. Never hard-code copy in components.
- Plan: docs/phased-plan.md — tick a phase when done.
- Before any commit: npm run build must pass.
- Every change adds a line to CHANGELOG.md.
- Lessons: (add each gotcha here, one line)
```

Long procedures go in skills (`.claude/skills/`). A rule with zero exceptions goes in a hook.



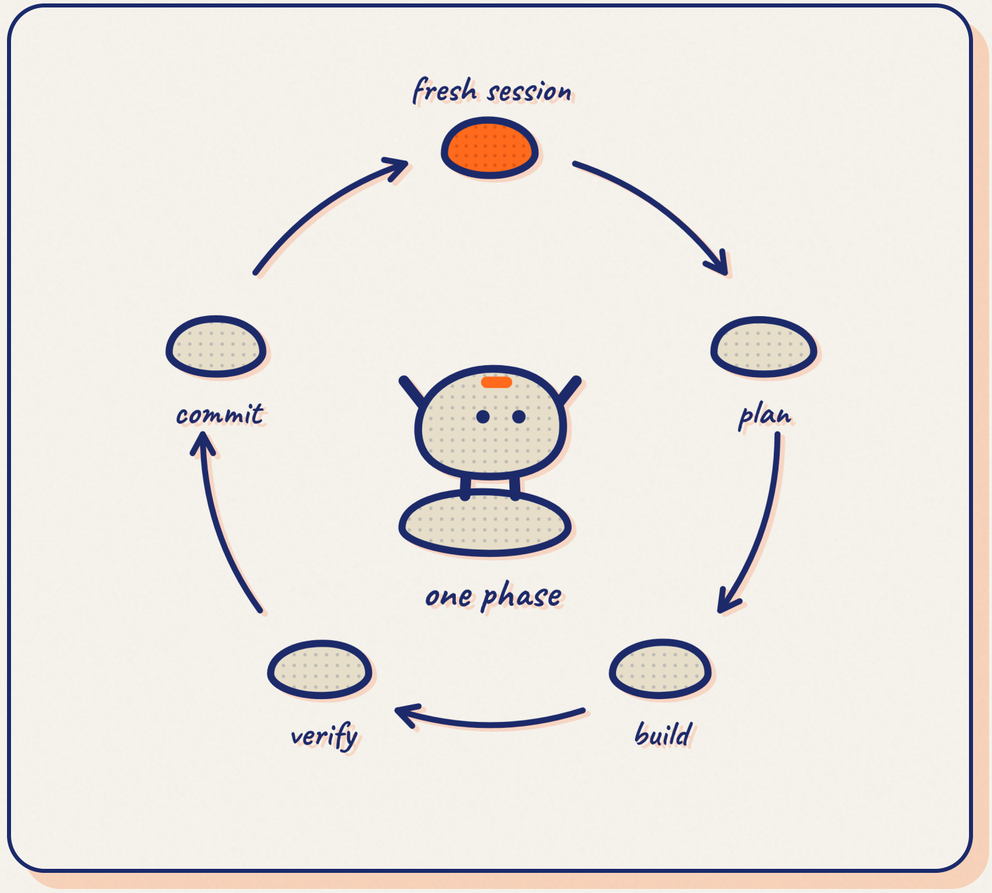
IN PLAIN WORDS **CLAUDE.md** a note Claude reads every session · **skill** a saved how-to it opens only when needed · **hook** a rule enforced automatically.

04 · BUILD 1:50

One phase = one session

Context is the constraint: Claude gets less accurate as a conversation fills up. A fresh session per phase keeps it small; the docs carry everything across.

1. **Fresh session** — new chat, or `/clear`
2. **Plan mode** — `Shift`+`Tab`; read the plan, approve it
3. **Build** — let it work; `Esc` to steer, `/rewind` to undo
4. **Verify** — build, run, look (slide 21)
5. **Commit** + **push**, then tick the phase in the plan



IN PLAIN WORDS **Context** how much of the chat Claude can hold in mind · **/clear** start a fresh chat · **plan mode** Claude proposes, you approve, then it edits.

04 · BUILD 1:55 · PHASE 1 NOW

The phase prompt

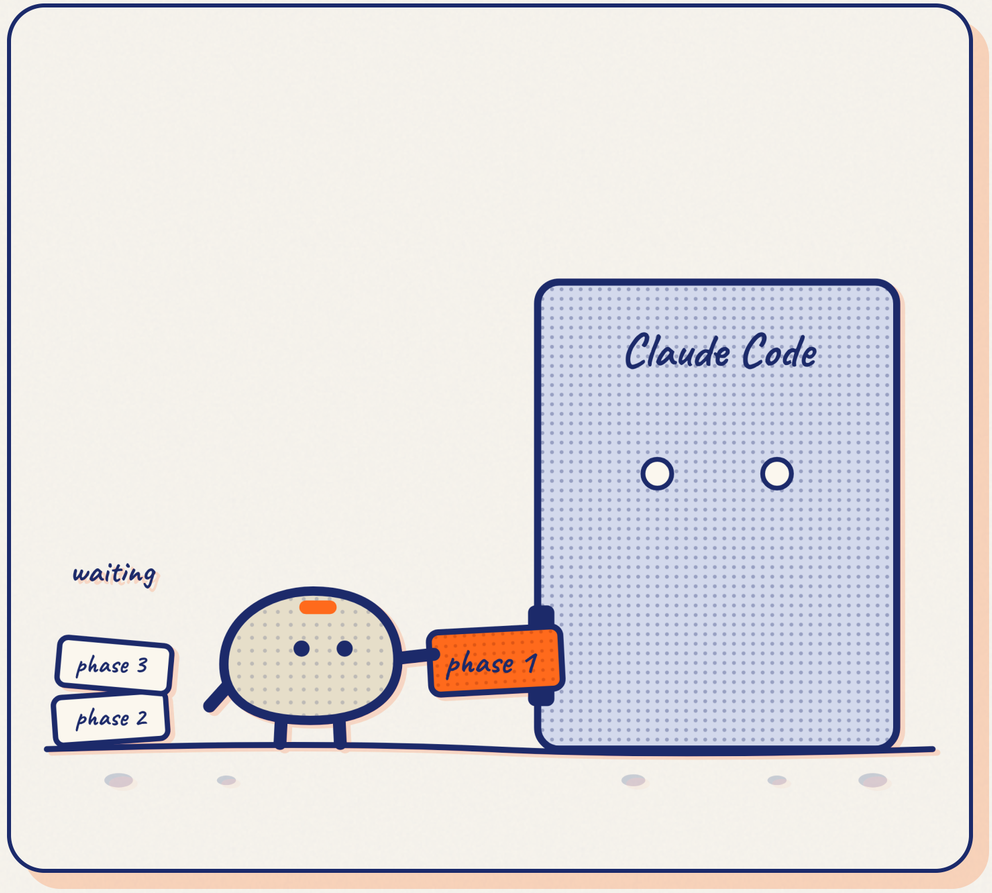
PROMPT · CLAUDE CODE, PLAN MODE

```
Read CLAUDE.md and docs/phased-plan.md. We are doing Phase 1 only.

First make a plan: the files you'll create or change and how you'll
verify each feature. Ask me anything unclear before you start.

Keep content in content/ and read it only through src/lib/content.
When done: run the build, start the dev server and check every page
renders at phone and desktop width, add a line to CHANGELOG.md,
tick Phase 1 in the plan, and commit.
```

✓ YOU SHOULD NOW HAVE **Phase 1 running on localhost with your real content.**



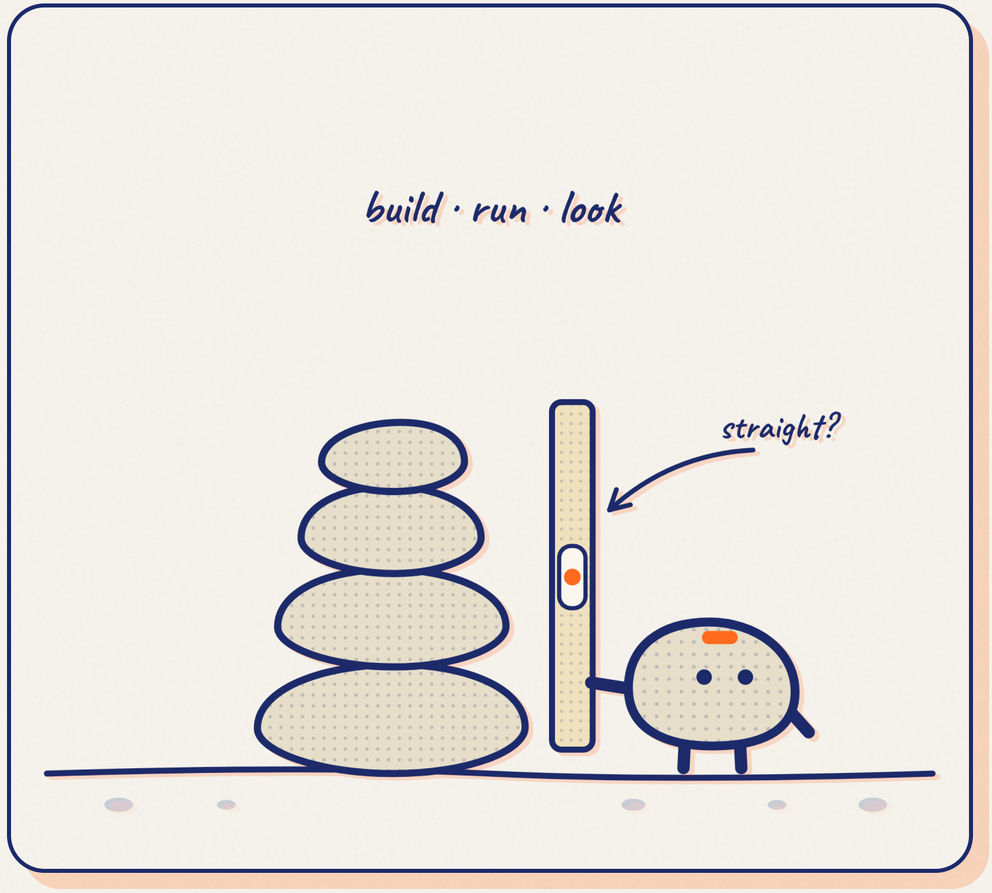
IN PLAIN WORDS **Dev server / localhost** a private preview running on your own laptop · **CHANGELOG.md** a running list of what changed.

04 · BUILD 2:05

Give Claude a way to check its work

- **The build passes** — the cheapest test there is.
- **It runs and it looks right** — ask Claude to open each page in a browser and screenshot it at phone width; `/verify` does this.
- **A second pair of eyes** — `/code-review` reviews the diff in a fresh session that never saw the reasoning.
- **You** — click through it on your own phone.

> **Done means deployed, and you looked at it.**



IN PLAIN WORDS **Build** turning your files into the finished site · **code review** a second reader checking a change before it ships.

04 · BUILD 2:10

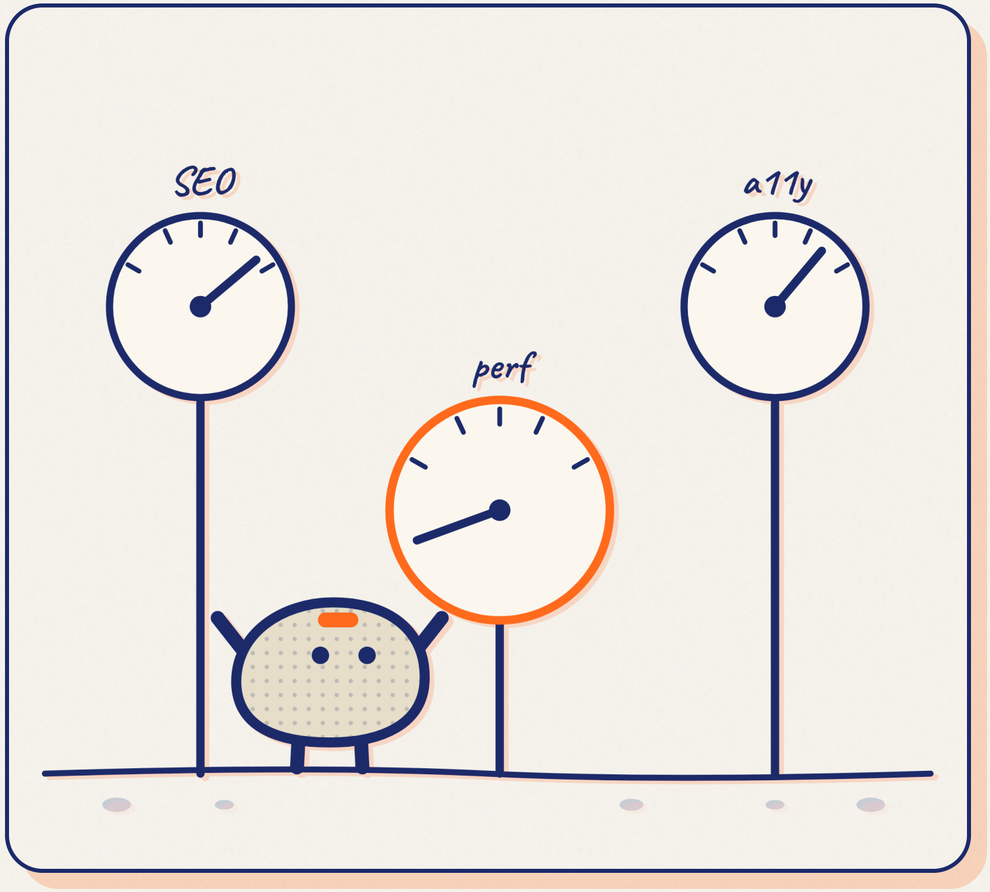
SEO, performance, accessibility

PROMPT · CLAUDE CODE

```
Audit the site for SEO, performance and accessibility. Build it and run
Lighthouse (mobile) on every page. Check each page has a unique title,
meta description, canonical URL and a 1200×630 Open Graph image; that
sitemap.xml and robots.txt exist; every image has alt text, width and
height, a modern format, under 200 KB; contrast and keyboard navigation.
Report a table ranked by impact, fix the top items, show before/after
scores.
```

Lighthouse ≥ 90 | LCP < 2.5 s | INP < 200 ms | CLS < 0.1

share card on every page



IN PLAIN WORDS **Lighthouse** Google's free report card · **LCP / INP / CLS** how fast the main content shows, how fast a tap responds, how much the page jumps · **alt text** image descriptions for screen readers.

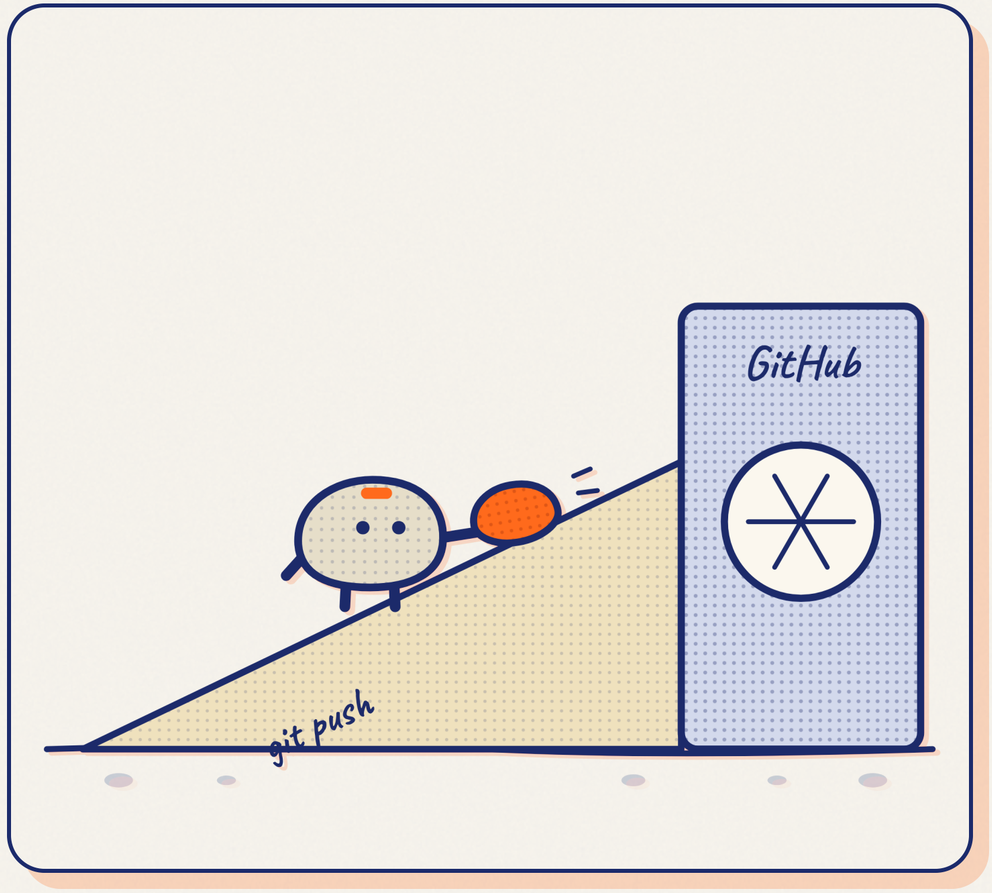
05 · SHIP 2:20

Push to GitHub

```
# one branch per phase
git checkout -b phase-2
git add -A
git commit -m "Phase 2: projects page"
git push -u origin phase-2
# open a pull request, check its preview, merge to main
```

Or just say it: *“commit this and push it to a new branch called phase-2”*. Claude Code writes the commit message from the diff.

✓ YOU SHOULD NOW HAVE your code on github.com, main branch, Phase 0 and 1 in the history.



IN PLAIN WORDS **Push** upload your checkpoints to GitHub · **branch** a safe copy to try a change · **pull request** asking to merge that copy, with a review page.

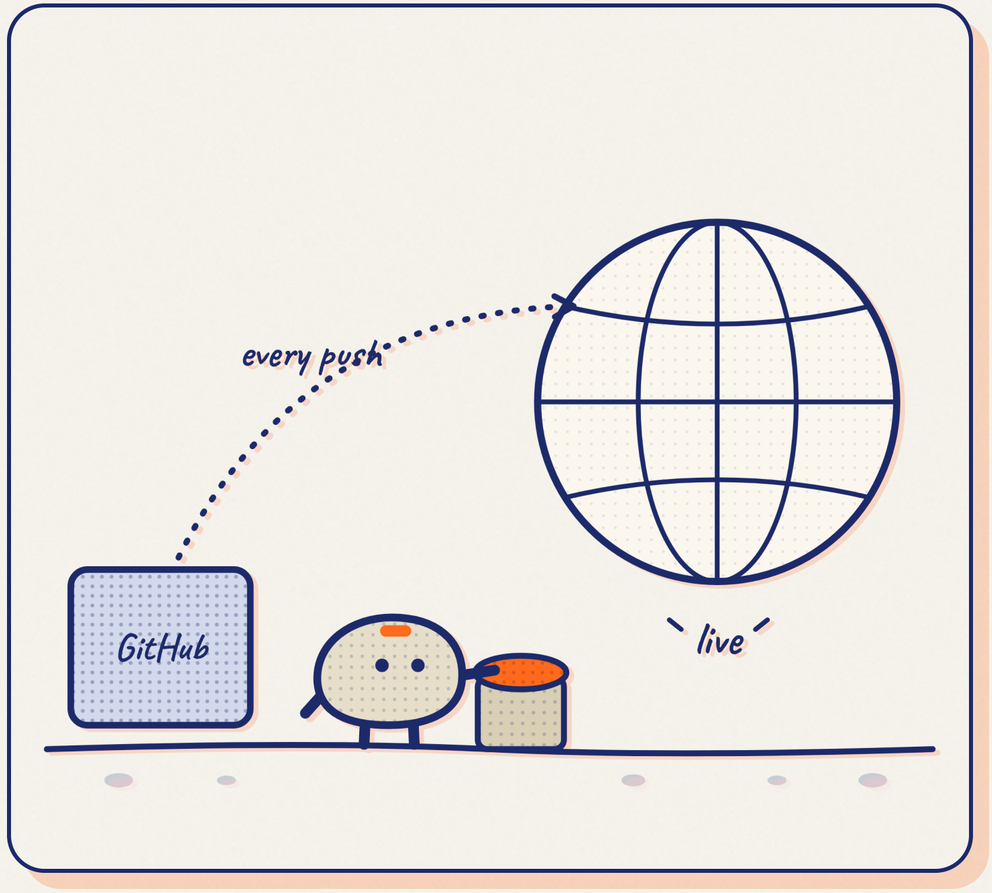
05 · SHIP 2:25

Deploy on Cloudflare

1. dash.cloudflare.com → **Workers & Pages** → **Create**
2. **Import a repository** → connect GitHub, grant only this repo
3. Build command `npm run build`; output `dist` (Astro) — most frameworks are detected
4. Environment variables, if any. Secrets never go in the repo.
5. **Deploy** → live at `your-site.<you>.workers.dev`

From now on, a push to **main** is production and every other branch gets a **preview URL**.

Cloudflare puts new features into Workers with static assets; Pages still works (the case study runs on Pages).



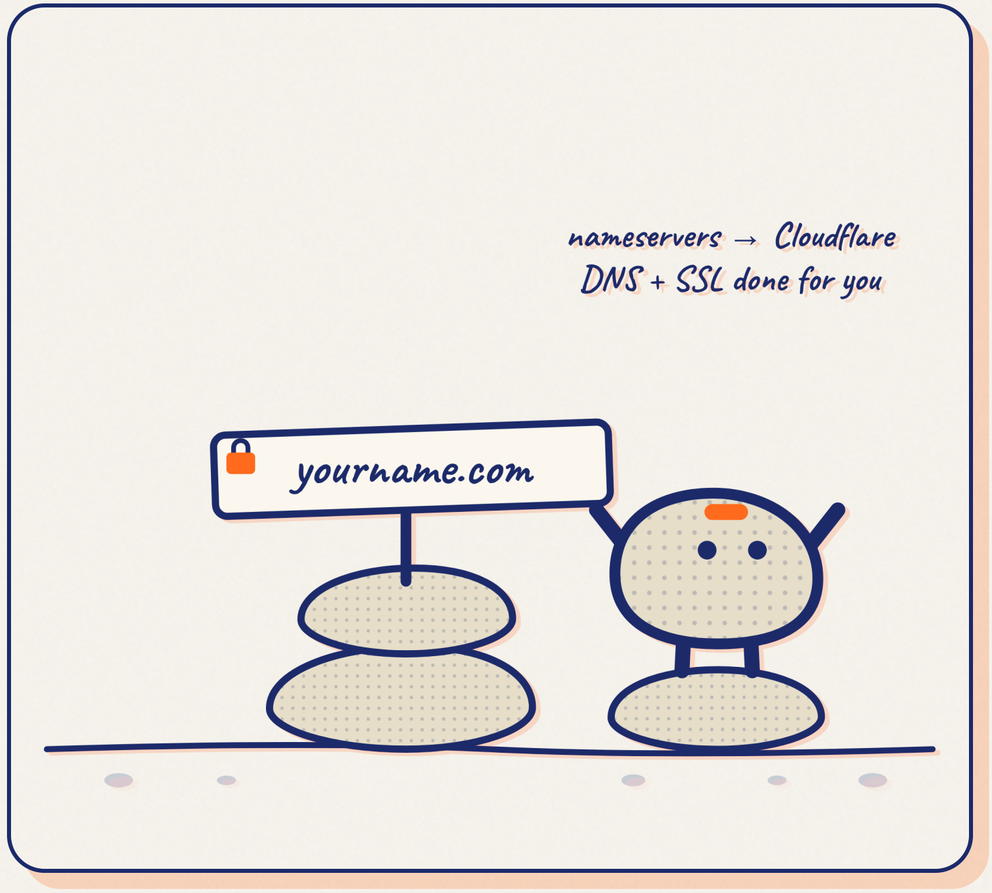
IN PLAIN WORDS **Deploy** put the built site online · **build command** the instruction that makes the files · **preview URL** a test link per branch · **environment variable** a setting kept out of the code.

05 · SHIP 2:35

Your domain and DNS

1. **Buy a domain** — Cloudflare Registrar sells at cost, or any registrar
2. Bought elsewhere? **Add the site** to Cloudflare (free plan) and set the registrar's **nameservers** to the two Cloudflare gives you. Wait for “Active”.
3. Your Worker → **Settings** → **Domains & Routes** → **Add** → **Custom domain**. Cloudflare creates the DNS record and the certificate.
4. Add `www` too, and redirect it to the bare domain with a Redirect Rule
5. Open `https://yourname.com` on your phone

> ✓ YOU SHOULD NOW HAVE your site on your own domain, over HTTPS. Nameserver changes can take a few hours — finish at home if needed.



IN PLAIN WORDS **Domain** your site's name · **DNS** the internet's phonebook · **nameservers** who keeps your entry in it · **SSL / HTTPS** the padlock: traffic is encrypted.

06 · CASE STUDY 2:38

sankettambare.in

A personal site built and grown this way: books, races, treks, projects, writing, a newsletter, and a chatbot over all of it.

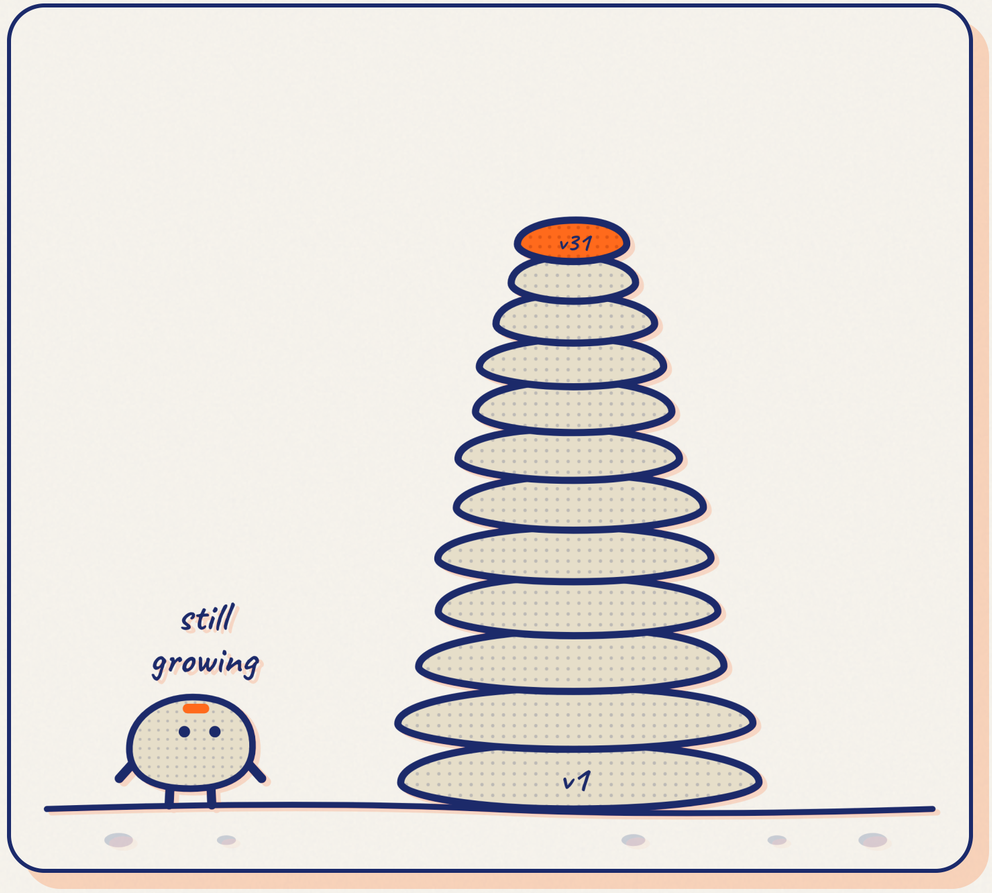
React 18 + Vite · Tailwind · Cloudflare Pages + Functions · GitHub · Claude Code

31 major versions

43 routes, each with a share card

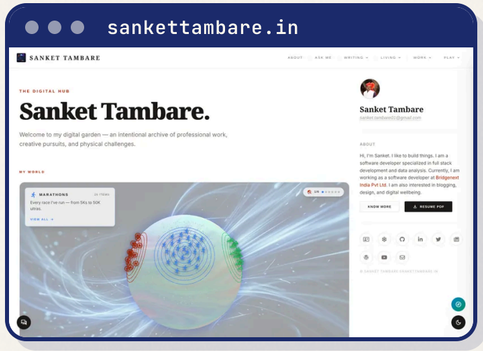
30 docs Claude reads before touching a part

It has a database too (Supabase). We skip it today: start with files in `content/`. Because data and code are apart, moving later is one module's job.

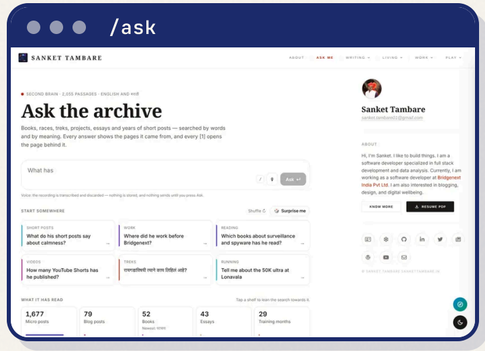


IN PLAIN WORDS **Pages Functions** small programs that run on Cloudflare when someone visits · **database** an organised store for many records — skipped today.

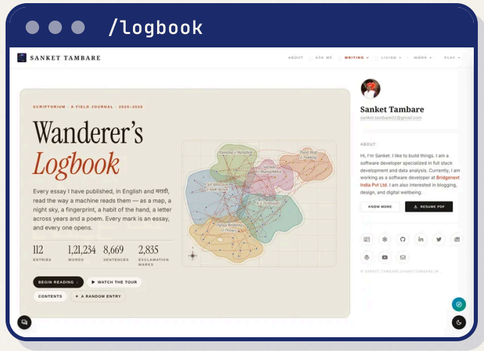
What a personal site can become



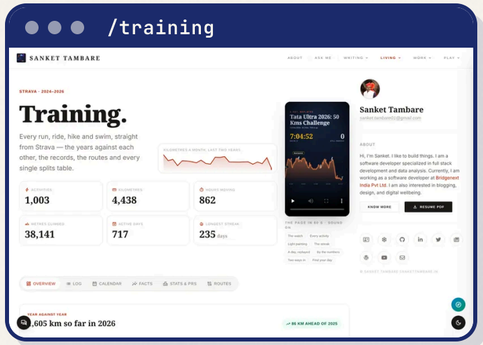
A world map of you
Every trek, race and post placed on one globe.



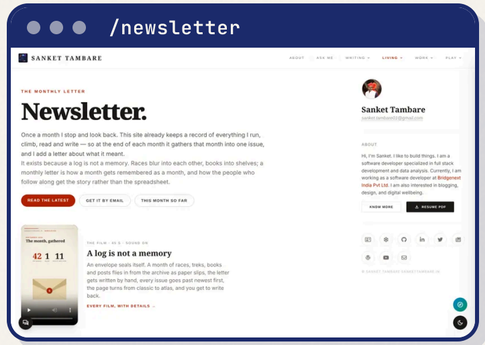
A chatbot over your archive
Answers from your own posts, with sources.



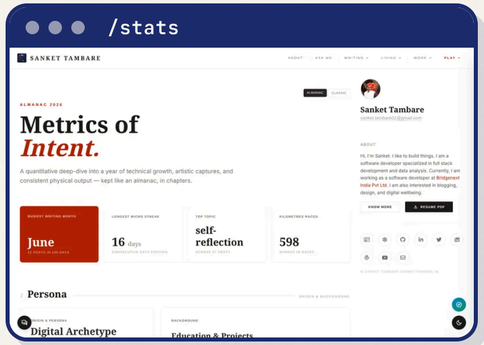
Your writing, as data
Every essay, mapped by meaning.



Your training log
Every Strava activity, records and routes.



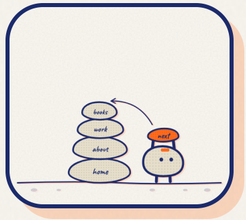
A monthly newsletter
Drafted from what the site recorded.



A yearly almanac
Your numbers, computed, never typed.

Start with four pages. Then add a room at a time:

- Books you've read, with notes
- Races, treks, travel — with photos
- Projects as case studies
- Talks and decks, embedded
- Videos from your channel
- A "now" page and a changelog
- Tags that link it all together
- A share card for every page



06 · CASE STUDY 2:42

What kept it iterating

ONE SOURCE PER FACT

Name, bio, links in one file; every page and crawler reads it.

CONTENT APART FROM CODE

Components read through one data module, never raw.

A CHANGELOG ON EVERY CHANGE

Same commit, same diff — 31 majors of readable history.

TESTS THAT GUARD THE RULES

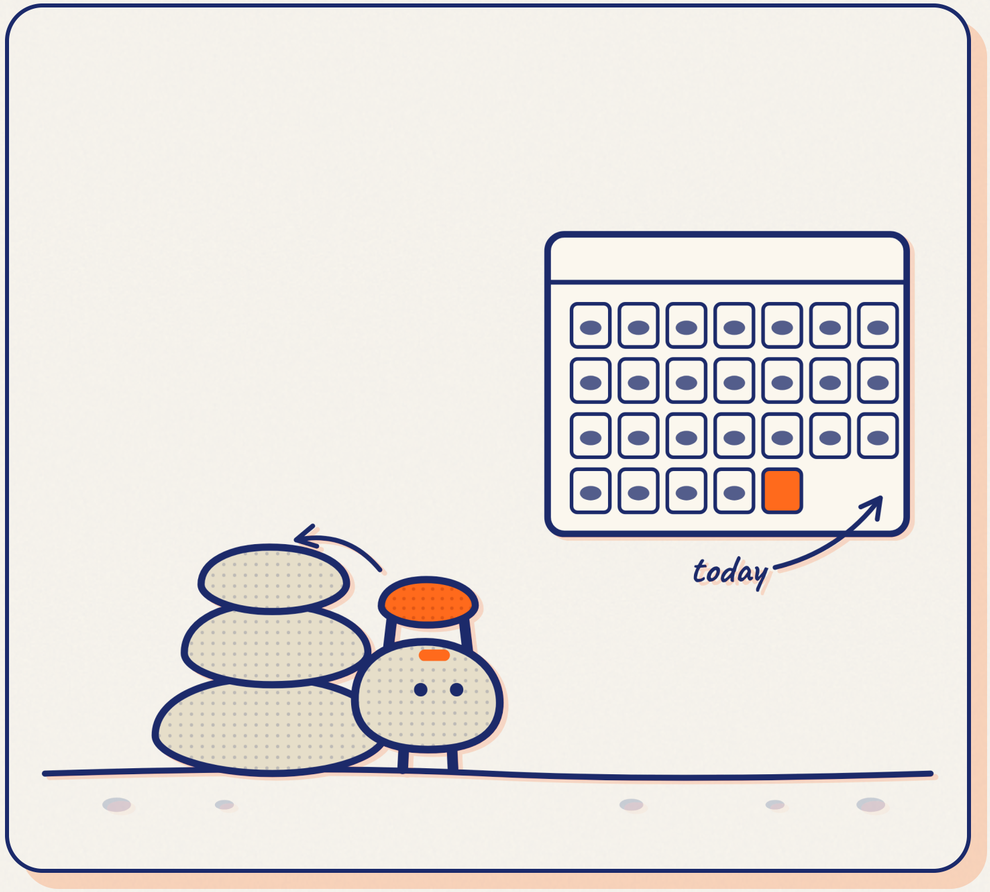
A new route with no share card fails the build.

GENERATED, NEVER HAND-EDITED

Sitemap, feeds, docs — rebuilt from the source.

LESSONS WRITTEN DOWN

Each gotcha is a line in CLAUDE.md, so no session repeats it.



IN PLAIN WORDS **Source of truth** the one place a fact is written · **test** an automatic check that fails if a rule is broken · **generated** made by a script, never by hand.

06 · CASE STUDY 2:44

What broke, and what it taught

BLANK TO CRAWLERS

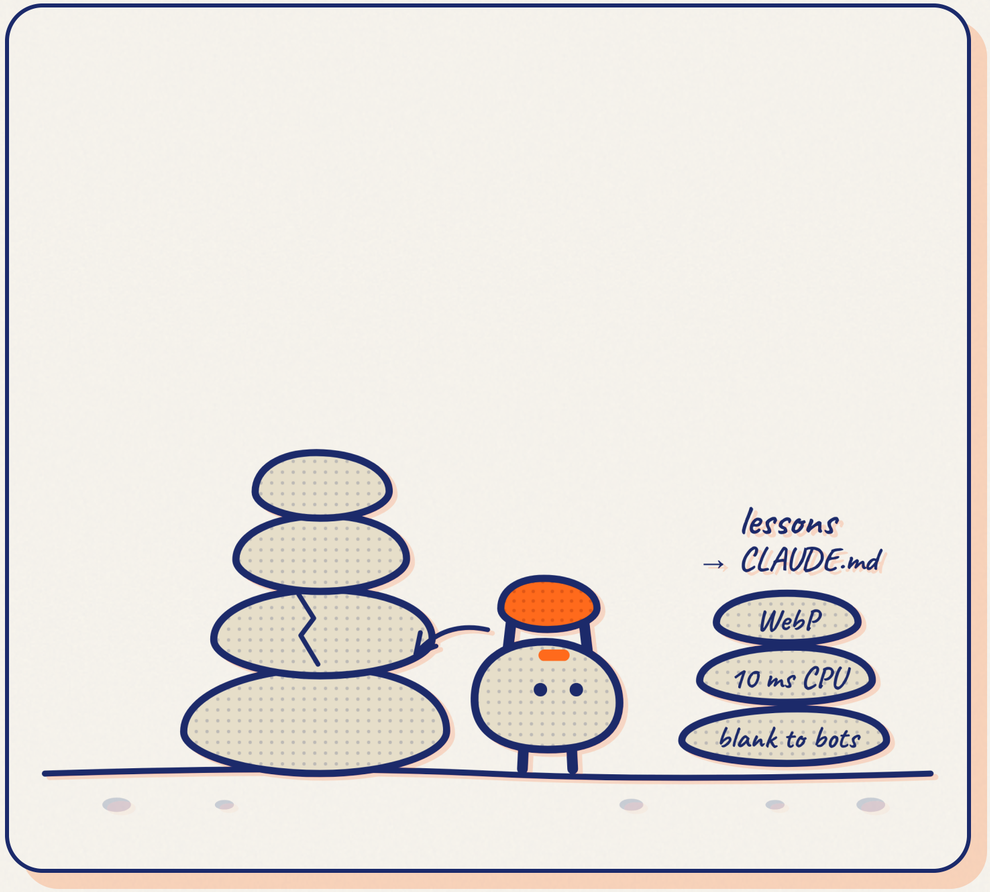
A React app renders in the browser; GPTBot, ClaudeBot and most crawlers don't run JavaScript. Fix: an edge middleware writes each page as plain HTML. **Lesson: pick a framework that renders HTML, or plan for it on day one.**

10 MS OF CPU

Share images were drawn per request at the edge; one took ~400 ms, so production returned errors and links unfurled with no image. Fix: render them at build time and commit the PNGs. **Lesson: local dev doesn't enforce limits — test on the real deploy.**

SILENT WEBP

The card renderer drew nothing for WebP photos, with no error. **Lesson: write the gotcha in CLAUDE.md the day you find it.**



IN PLAIN WORDS **Renders in the browser** the page is assembled by JavaScript after loading, so robots that don't run it see a blank page · **WebP** a modern image format.

Visiting Card

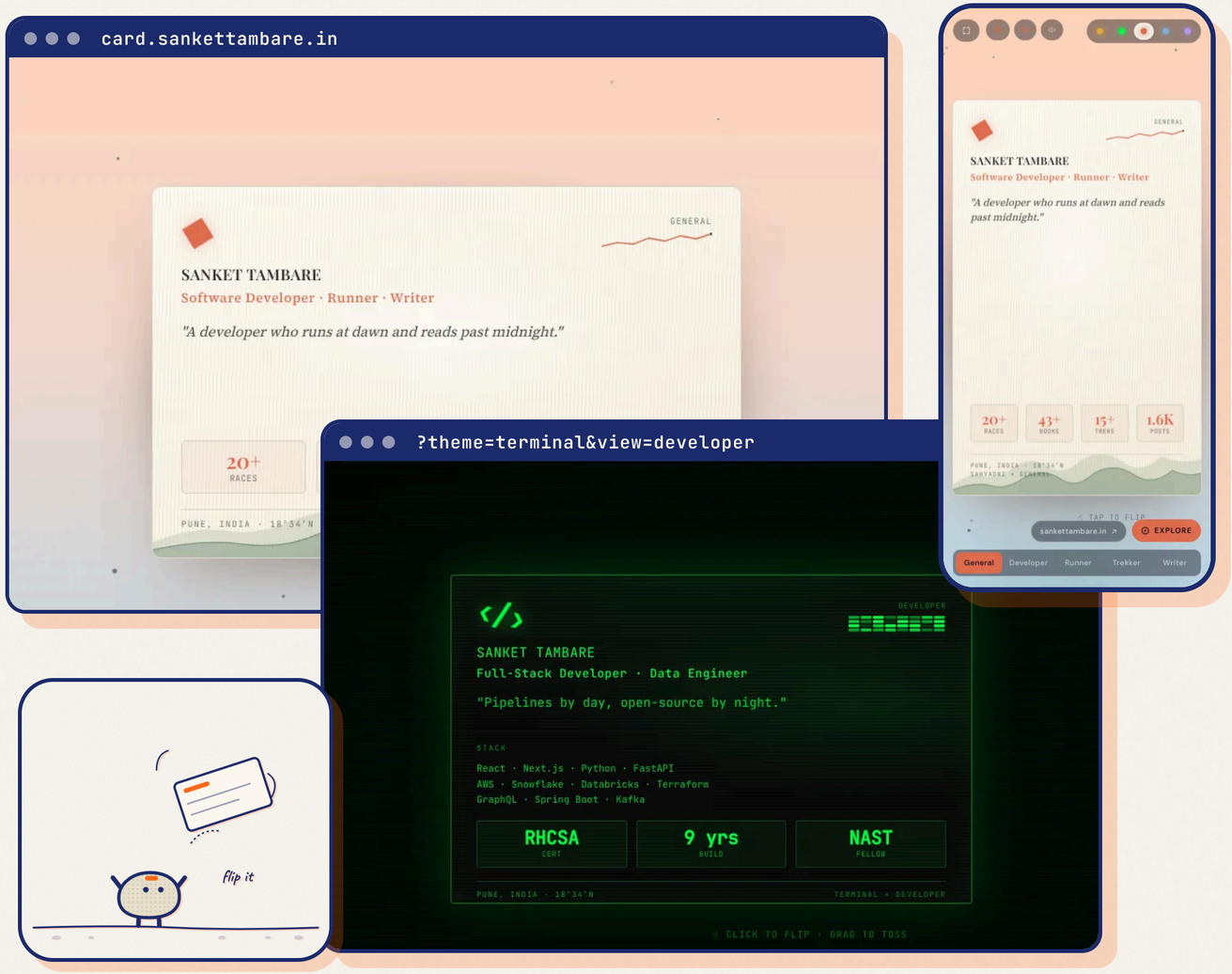
An interactive visiting card. Flip it, toss it, pick the version of you that fits the person you're handing it to.

- **Five themes** — Wanderer, Terminal, Sahyadri, Blueprint, Holographic
- **Five views** — General, Developer, Runner, Trekker, Writer
- **The link carries the choice** (`?theme=terminal&view=developer`), so each audience gets its own URL
- Download a vCard, copy the details, open the résumé; motion and sound toggles

`Vite` `TypeScript` `Cloudflare Pages` `Claude Code`

The perfect first project: small, personal, and live in an evening.

→ `card.sankettambare.in`



IN PLAIN WORDS **vCard** a contact file your phone saves in one tap · **URL parameter** the ?theme=... part of a link that carries a choice.

Vetal Tekdi Birding Marathon

A bilingual event site for the Vetal Tekdi Bachav Kruti Samiti: one week, one hill, every bird counted — and every sighting becomes evidence for saving Pune's last wild hill.

164
species in one week, 2025

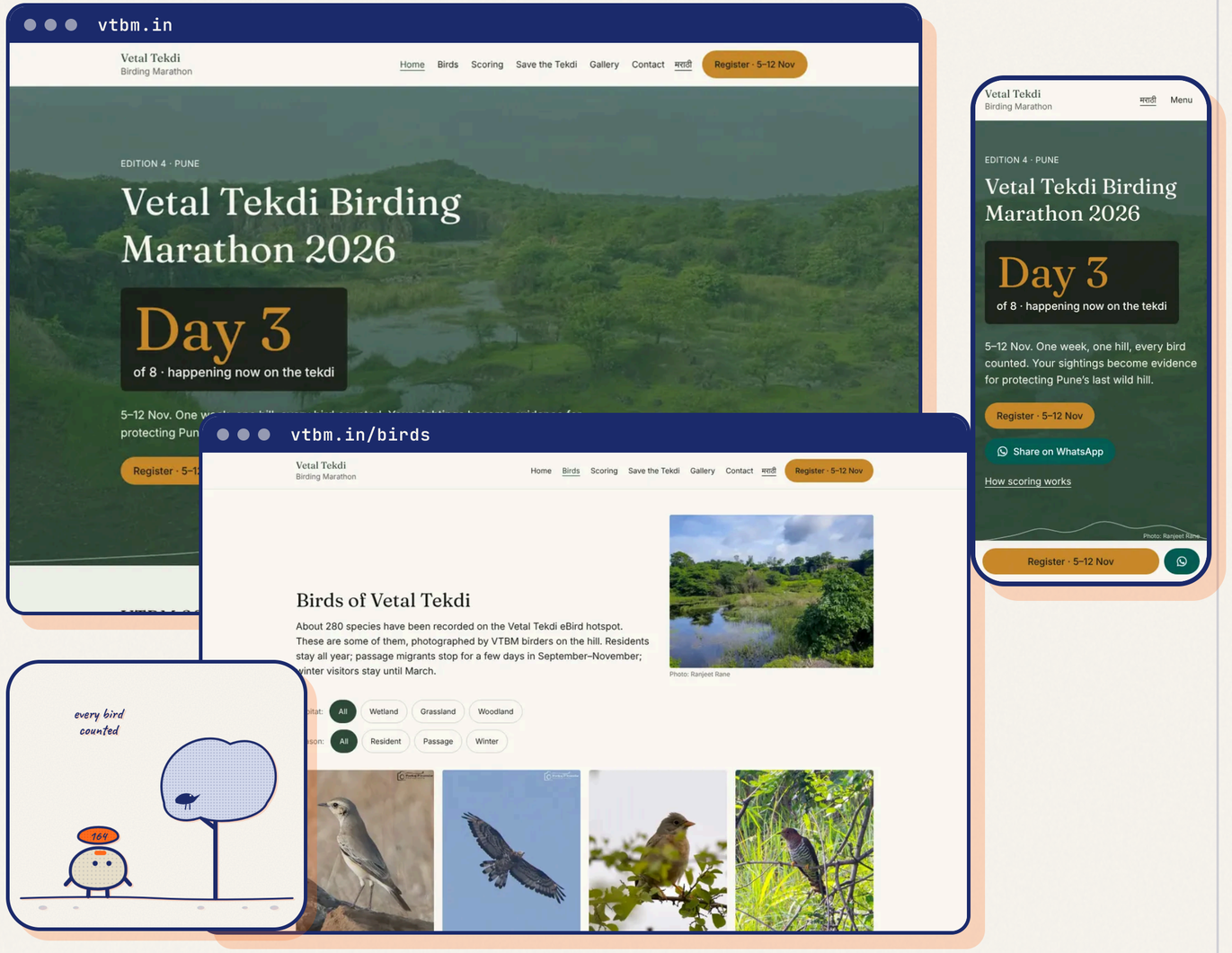
92
eBird checklists

111→164
species per edition, 2023–25

- **Live from eBird** — the species seen on the hill this week, with photos
- A bird guide filtered by habitat and season; scoring; registration
- English and Marathi, fast on a phone on the hill

Astro · Tailwind · Cloudflare Pages + Functions · eBird API

→ vtbm.in



IN PLAIN WORDS **API** a way for one site to ask another for data — here, eBird's sightings · **bilingual** every page in English and Marathi.

RunFolio

The portfolio for runners — every finish line, worth framing. Race photos, splits and PBs turned into one public profile and one link.

- **Log any race** — distance, time, bib and photos; road, trail or track
- **Share one link** that reads like a magazine spread — ledger, gallery or stats, and the URL carries the choice
- Free tools for runners; Strava sync coming back soon
- A real product: sign-in, profiles, image storage

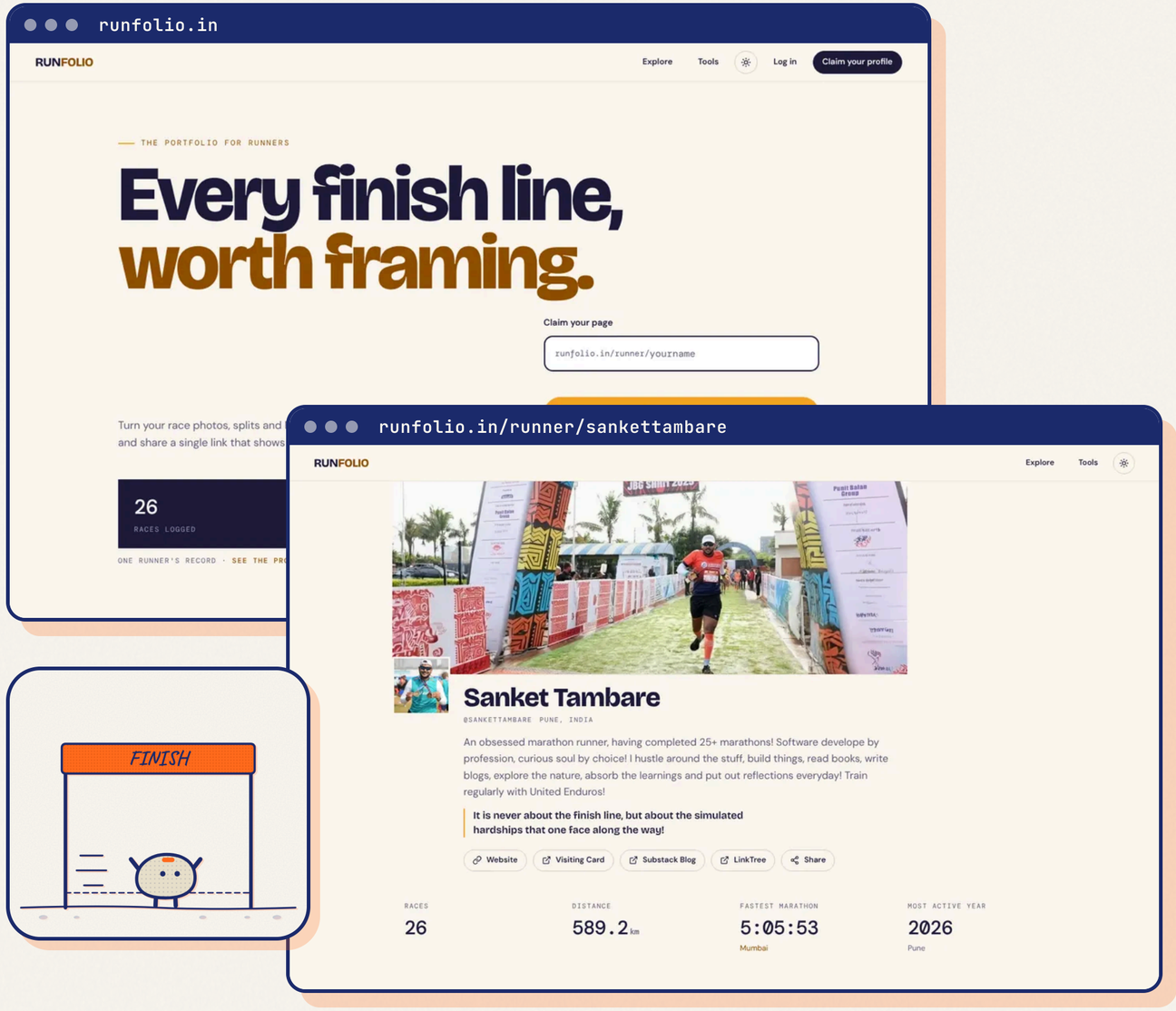
26
races on my own profile

589 km
of racing logged

free
for every runner

Astro · Cloudflare Workers · Supabase · R2

→ runfolio.in



IN PLAIN WORDS **Supabase** a hosted database with sign-in built in · **R2** Cloudflare's storage for photos · **PB** personal best.

Pumped Storage: are they useful?

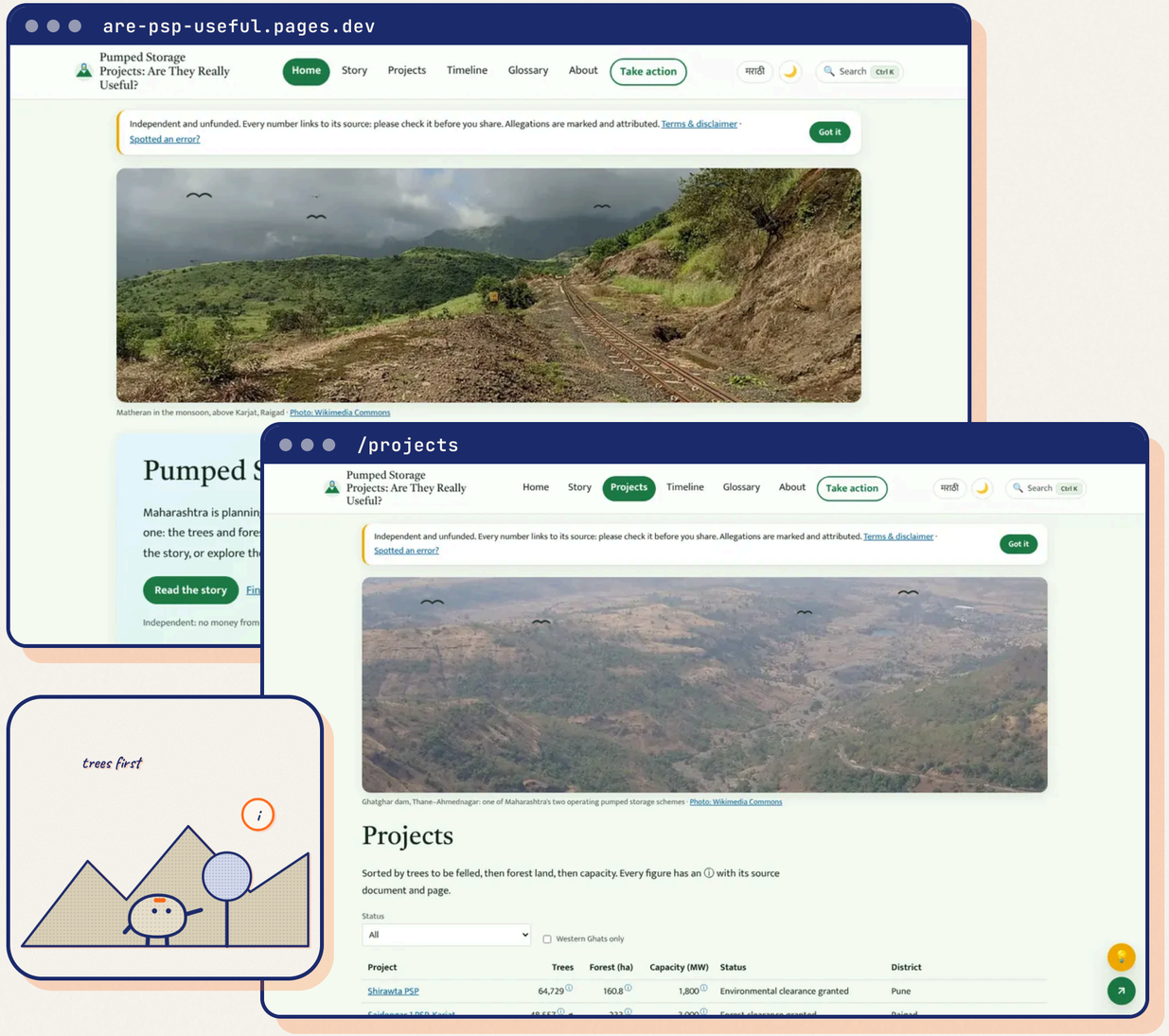
An independent, bilingual data story on Maharashtra's push for pumped storage projects in the Sahyadri — the trees and forest first, then the clearances and the people.

- **Every number has a source** — an ⓘ bubble quotes the document and page
- **Projects sorted by trees to be felled,** with forest land and capacity
- A story, a timeline, a glossary, a map; a daily news sync
- Open data downloads; English and Marathi

`Astro 5` `Cloudflare Pages` `sources, not opinions`

Proof the method isn't only for portfolios.

→ `are-psp-useful.pages.dev`



IN PLAIN WORDS **Data story** an article built on figures you can check · **open data** the numbers offered as downloads.

08 · ITERATE 2:52

From now on, a change is a prompt

LAPTOP

Claude Code in the terminal, VS Code or the desktop app.

BROWSER

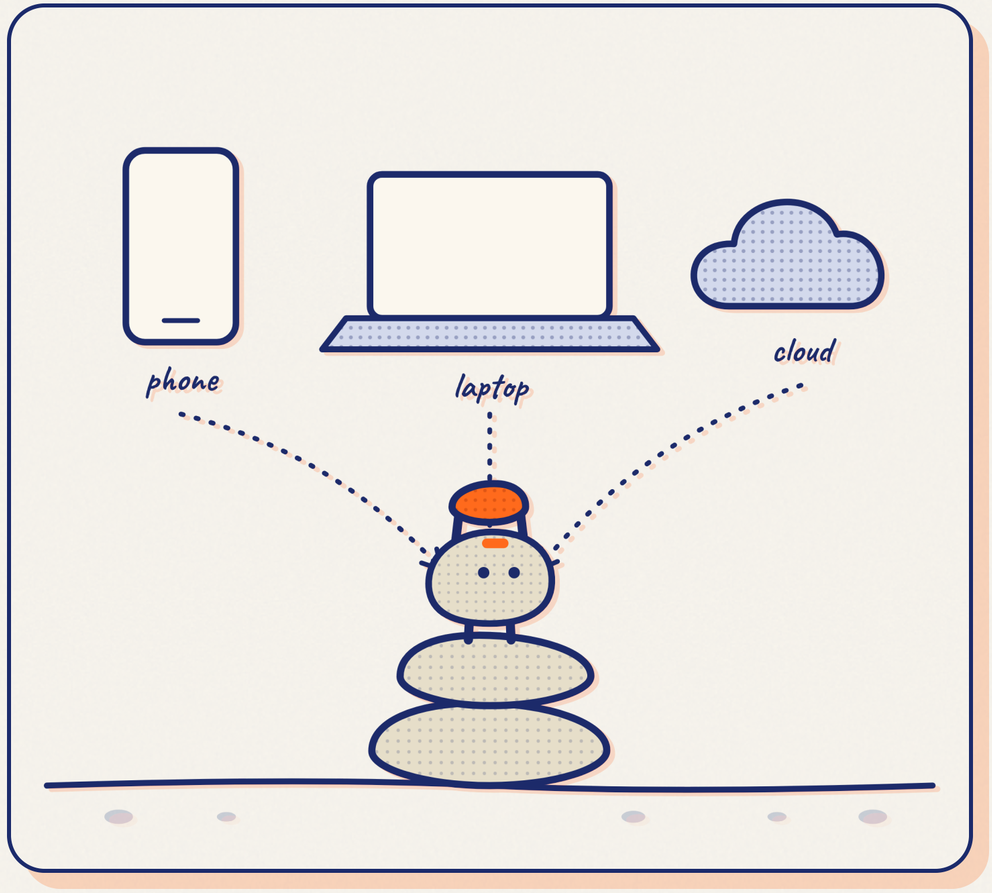
Claude Code on the web runs in the cloud on your GitHub repo.

PHONE

The Claude app: describe it on the train, review the preview later.

prompt → branch → preview URL → merge → live. The next iterations are about UI and content: one page, one section, one photo at a time.

- Weekly: add what you did
- Monthly: run the audit prompt
- Quarterly: redesign one page, not the site



IN PLAIN WORDS **Claude Code on the web** Claude Code running in the cloud on your GitHub repo — nothing to install.

08 · ITERATE 2:54 · HOMEWORK

Seven days, seven stones

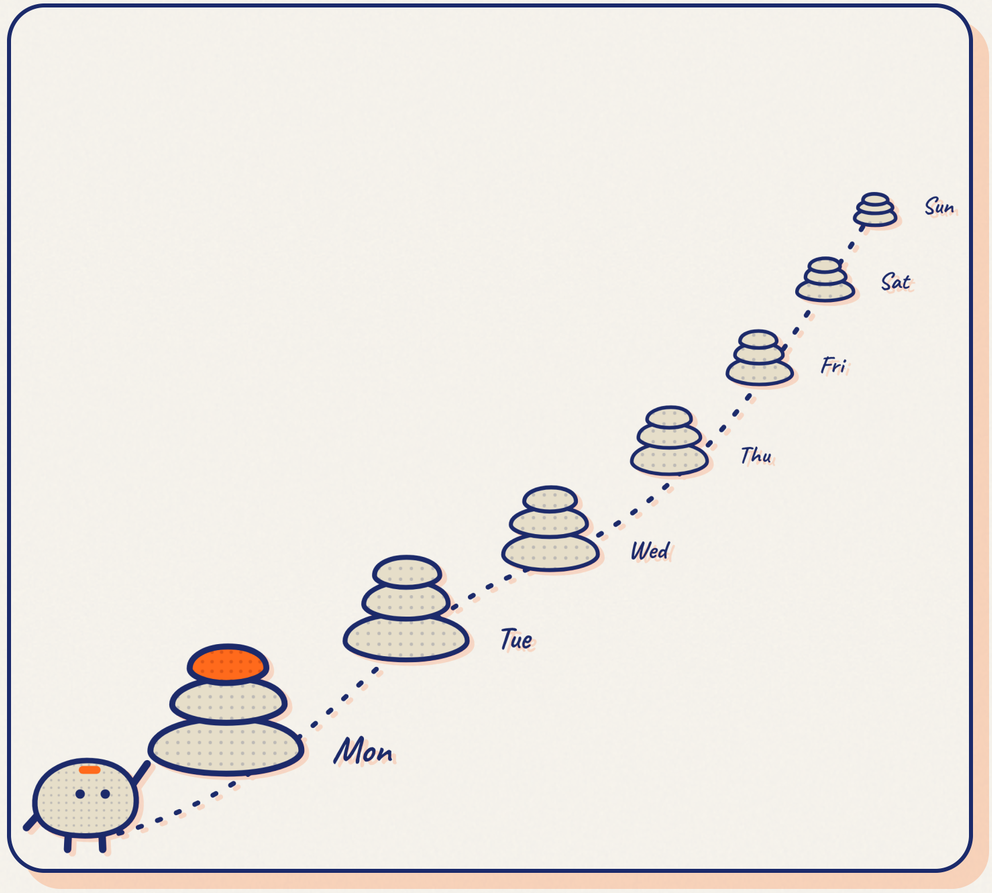
1. Live on workers.dev
2. Your own domain
3. Real content on every page
4. Run the audit prompt
5. One new section
6. A friend uses it on their phone — fix the first thing
7. Write about building it

Take-aways

The prompt pack — every prompt from today
Doc templates — idea, concept, decisions, plan, CLAUDE.md
This deck as a PDF

Read

Claude Code best practices
Astro content collections
Cloudflare Workers static assets
Cloudflare custom domains
Core Web Vitals · Google SEO starter guide



IN PLAIN WORDS **workers.dev** the free address Cloudflare gives your site before you have a domain.

09 · CLOSE 2:56 · YOUR TURN

How was it?

1 · OVERALL

How was it, 1 to 5?

And one word for today.

2 · THE LEARNING

What stood out most?

The one idea you'll use tomorrow.

3 · YOUR SITE

How far did you get?

Docs, repo, workers.dev, your domain?

4 · FRICTION

Where did you get stuck?

The step that needs more time or a better prompt.

5 · NEXT TIME

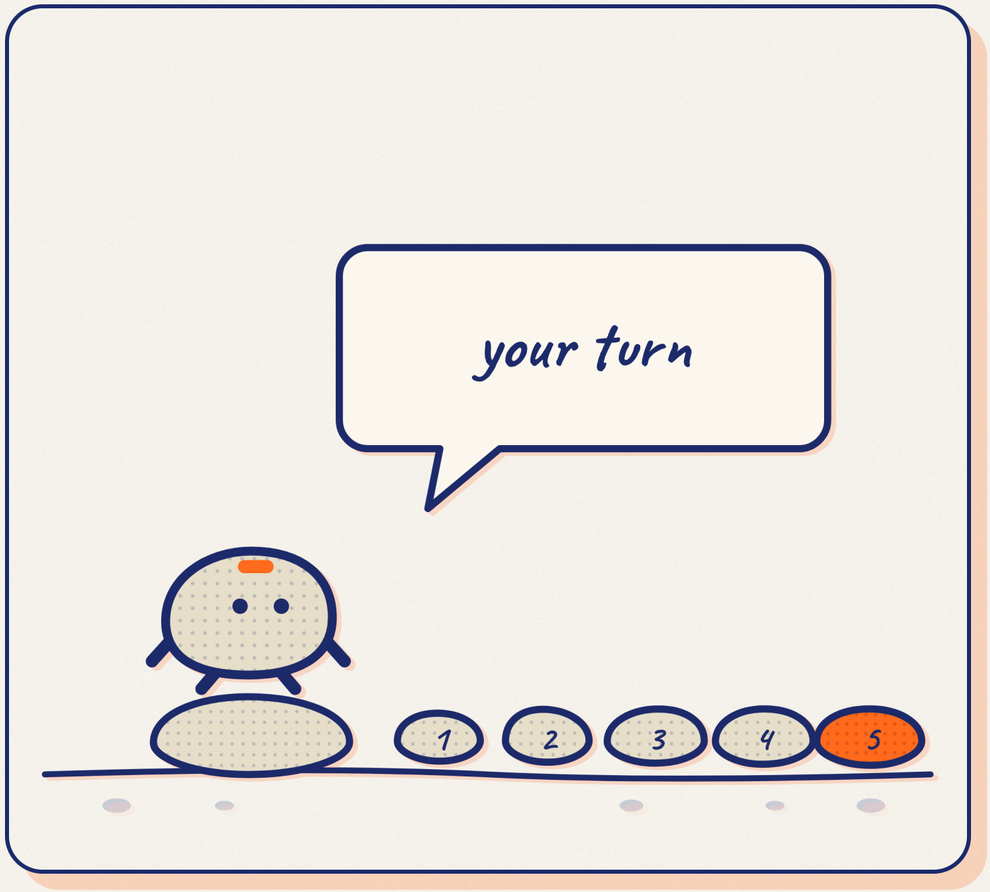
What should I add or cut?

A topic you wanted, a part that dragged.

6 · SHARE

Send me your URL

By Sunday — I'll look at every one.



09 · CLOSE 2:59

Thank you

Add one stone a day. Send me what you build.

WEBSITE
sankettambare.in

VISITING CARD
card.sankettambare.in

LINKEDIN
in/sankettambare

GITHUB
daredavil01

SUBSTACK
sankettambare.substack.com

X
@i_daredavil

EMAIL
sanket.tambare01@gmail.com

THE PROMPT PACK
every prompt from today

Scan for my card —
save the contact,
every link in one place.